

Table of Contents

[TDEDefTopic](#)

[tdtde.1.02](#)

[tdtde.1.03](#)

[tdtde.1.04](#)

[tdtde.1.05](#)

[tdtde.1.06](#)

[tdtde.1.07](#)

[tdtde.1.08](#)

[tdtde.1.09](#)

[tdtde.1.10](#)

[tdtde.1.11](#)

[tdtde.1.12](#)

[tdtde.1.13](#)

[tdtde.1.14](#)

[tdtde.1.15](#)

[tdtde.1.16](#)

[tdtde.1.17](#)

[tdtde.1.18](#)

[tdtde.1.19](#)

[tdtde.1.20](#)

[tdtde.1.21](#)

[tdtde.1.22](#)

[tdtde.1.23](#)

[tdtde.1.24](#)

[tdtde.1.25](#)

[tdtde.1.26](#)

[tdtde.1.27](#)

[tdtde.1.28](#)

[tdtde.1.29](#)

[tdtools.2.01](#)[tdtools.2.02](#)[tdtools.2.03](#)[tdtools.2.04](#)[tdtools.2.05](#)[tdtools.2.06](#)[tdtools.2.07](#)[tdtools.2.08](#)[tdtools.2.09](#)[tdtools.2.10](#)[tdtools.2.11](#)[tdtools.2.12](#)[tdtools.2.13](#)[tdtools.2.14](#)[tdtools.2.15](#)[tdtools.2.16](#)[tdtools.2.17](#)[tdtools.2.18](#)[tdtools.2.19](#)[tdtools.2.20](#)[tdtools.2.21](#)[tdtools.2.22](#)[tdtools.2.23](#)[tdtools.2.24](#)[tdtools.2.25](#)[tdtools.2.26](#)[tdtools.2.27](#)[tdtools.2.28](#)[tdtools.2.29](#)[tdtools.2.30](#)[tdtools.2.31](#)[tdtools.2.32](#)

[tdtools.2.33](#)
[tdtools.2.34](#)
[tdtools.2.35](#)
[tdtools.2.36](#)
[tdtools.2.37](#)
[tdtools.2.38](#)
[tdtools.2.39](#)
[tdtools.2.40](#)
[tdtools.2.41](#)
[tdtools.2.42](#)

TDEDefTopic

	
---	---

Test Debug Environment (TDE)

Test Debug Environment (TDE)

The IG-XL Test Debug Environment (TDE) is a workspace that integrates tools that you use to debug a test. These include general test system debug tools, as well as debug displays for specific instruments and instrument subsystems. By offering a set of integrated tools and providing a focused view of a test, TDE makes debugging an integral part of the test development process. Each tool available in TDE has its own help. This section includes topics concerning general TDE issues, such as managing tool windows in the TDE workspace. The following topics are available:

- | |
|---|
| <ul style="list-style-type: none">Using TDE |
| <ul style="list-style-type: none">Using Debug Displays in TDE |

- [Producing Code from Debug Displays](#)

- [Debugging in TDE](#)

- [Standard TDE Commands and Buttons](#)

TDE information also includes the following section:

- [TDE Tools](#)

This section on TDE tools describes the following tools:

- [Code Viewer](#)

- [FlowCallStack](#)

- [MiniBin](#)

- [MiniFlow](#)

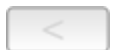
- [PSet and Signals Editor](#)

- [Results Tool](#)

- [Tester Summary Display](#)

- [VBT Debugger](#)

- [Watch](#)



	2015 Teradyne, Inc. All rights reserved.	
--	--	--

tdtde.1.02

<	>
-----------------	-----------------

Test Debug Environment (TDE) : Using TDE

Using TDE

The following topics are included here:

- | | |
|---|--|
| • | Invoking and Filling TDE |
| • | Using Toolgroup Windows |
| • | ToolSets |

See also [Standard TDE Commands and Buttons](#).

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

tdtde.1.03

	
---	---

Test Debug Environment (TDE) : Using TDE : Invoking and Filling TDE

Invoking and Filling TDE

Invoke TDE from DataTool. On the IG-XL Tools tab, in the One-Click Launch group, click Test Debug Environment\

When you invoke TDE in this way, TDE comes up empty. You have the following options for filling TDE to begin your session:

- Bring up tools or debug displays individually. On the TDE menu:
 - Tools accesses all the tools
 - Instruments accesses the debug displays for all instruments
 - Subsystems accesses the debug displays for all boards and subsystems
-  The Restore Last Session button or command (on the TDE menu) restores all tools that were open when you ended your last TDE session.
-  The Auto Display button or command (on the TDE menu) brings up a debug display for all instruments defined in the current channel map. Auto Display is enabled only after a test program is validated.
- Bring up a toolset, which is a named configuration of tools that you set up in a previous session. The menu to the right of the Master Site control contains the names of all the

currently defined toolsets. Selecting a toolset brings up the tools defined for that toolset. See [ToolSets](#).

The list of tools and instruments available in TDE is determined by your IG-XL installation and by the configuration file in use when you started TDE. If you don't see an instrument you think should be present, verify that the instrument is present in the configuration file.

Automatic TDE Invocation

In addition to explicit invocation, TDE also comes up in other circumstances.

You can invoke a TDE tool from outside of TDE. For example, you can invoke a debug display from the IG-XL Tools tab, in the Launch group, using the Debug Displays button, or you can invoke an instance editor from a Test Instances sheet. In this case the tool comes up in TDE.

TDE is also automatically invoked, in minimized form, when you run to a breakpoint in debug mode in DataTool. See [Debug Runs in DataTool and TDE](#). If you run to a breakpoint in VBT code, TDE is automatically invoked with the VBT Debugger. See [VBT Debugger](#).

			
2015 Teradyne, Inc. All rights reserved.			

tdtde.1.04



Test Debug Environment (TDE) : Using TDE : Using Toolgroup Windows

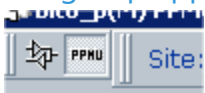
Using Toolgroup Windows

This section describes using toolgroups. A toolgroup is a window in TDE that contains one or more tools in stacked format. The topics included are:

- [Toolgroup Overview](#)
- [Recommendations for Using Toolgroups](#)
- [Adding a Tool to a Toolgroup](#)
- [Rearranging Tools and Toolgroup Windows](#)
- [Maximized View and Scrollbars](#)

Toolgroup Overview

In the TDE workspace, each window can be considered a toolgroup, where the toolgroup window contains one or more tools. If a toolgroup window contains more than one tool, the tools are stacked. That is, only the tool at the top of the stack is visible, and the icons for all the tools in the toolgroup appear at the left of the toolgroup window toolbar.



Tools Stacked in Toolset Window

For example, the icons in this toolbar indicate that the toolgroup window contains two debug displays, the Digital Channel Display and the PPMU Display. The PPMU, whose icon looks as though

it has been clicked, is currently selected. IG-XL assumes that you do not need to see all these tools at the same time.

Stacking the tools in a toolgroup conserves display space in TDE. Cycle through the tools in a single toolgroup by using the F6 key, or select a specific tool to display by clicking its icon. See [Recommendations for Using Toolgroups](#).

You create a toolgroup by adding one tool to another tool's window, as described in [Adding a Tool to a Toolgroup](#). You can arrange tools into toolgroup windows to suit your needs, as described in [Rearranging Tools and Toolgroup Windows](#). Then you can save your customized rearrangement as a toolset, so you can invoke it again in the future. See the description of [ToolSets](#). You should use the default positions, however, as described in [Recommendations for Using Toolgroups](#).

Recommendations for Using Toolgroups

TDE makes several tools readily available while making best use of the TDE display area. When using TDE, follow these recommendations:

- Within a toolgroup window, use F6 to cycle through the tools.

In a toolgroup window, only one tool is visible. To make a tool visible, you can click its icon in the toolgroup window toolbar. It is usually easier to use F6 (or Window>Next Tool) to cycle through the tools in the currently active toolgroup window.

To go to a specific tool in any toolgroup window, use the Window menu, which lists all the currently open tools.

- To work with a single tool, use Full Screen to maximize and restore it.



Although you can resize a window manually, you may find it easier to use Window>Full Screen command or button. The function toggles in and out of full screen mode for the currently active tool. In full screen mode, a tool takes up the entire TDE display area. You are most likely to need full screen mode for debug displays and for tools, such as PatternTool, that offer a lot of detail.

- To select a tool in TDE, use the Window menu.

The Window menu lists all the currently open tools in all the windows. To select a tool, select its name from the Window menu list.

- Use toolsets to save and restore desired configurations.

Once you have opened the tools you want, you can save the configuration to a named toolset. You can restore the toolset in a later session, or you can switch among toolsets in a single session. You

can store up to four toolsets. See [ToolSets](#).

Adding a Tool to a Toolgroup

To add a new tool to a another tool's window, select the window by clicking it. Then right-click one of the tool icons in the toolbar. The right-click menu contains the entries [Tools](#), [Instruments](#), and [Subsystems](#), each having a menu with the available items. Adding a tool in this way adds it to the current toolgroup window. In contrast, the same commands on the TDE command add the tools in the default positions, usually in separate windows.

You can also cut one window and paste it into another window, as described in the next section.

Rearranging Tools and Toolgroup Windows

You can rearrange the tools from their default positions using these commands in the [Window](#) menu:

Docking	Docks the current toolgroup window. The command has a pull-right menu with Left , Right , and Bottom . Select one of these to dock the toolgroup window to the specified side of the TDE window. Use the None option to undock the window.
Close 'toolname'	Closes the tool.
Detach 'toolname'	Removes the tool from this toolgroup window and moves it to a new toolgroup window by itself.
Cut 'toolname'	Removes the tool from this toolgroup window to let you paste it into a different toolgroup window. After cutting a tool, select an icon in the toolgroup window to which you want to paste the tool. Then right-click and select Paste Tool .
Paste Tool	Adds the cut tool to the current toolgroup window.

These commands apply to the currently selected toolgroup. The same commands are available if you right-click a tool's icon in the toolgroup window toolbar. For details, see [Window Menu](#).

Maximized View and Scrollbars

When a tool or debug display first comes up in TDE, it is in a regular-sized (that is, not maximized) tool window. You can have the tool or display expand to use the entire TDE window by using the [Maximize](#) control in the upper right corner of the tool window:



Maximize Control

The [Maximize](#) control is the square in the middle. But, if the TDE window is smaller than the maximized tool window, there are no scrollbars on the TDE window to let you scroll to see the rest of the debug display. This is because of an implementation constraint.

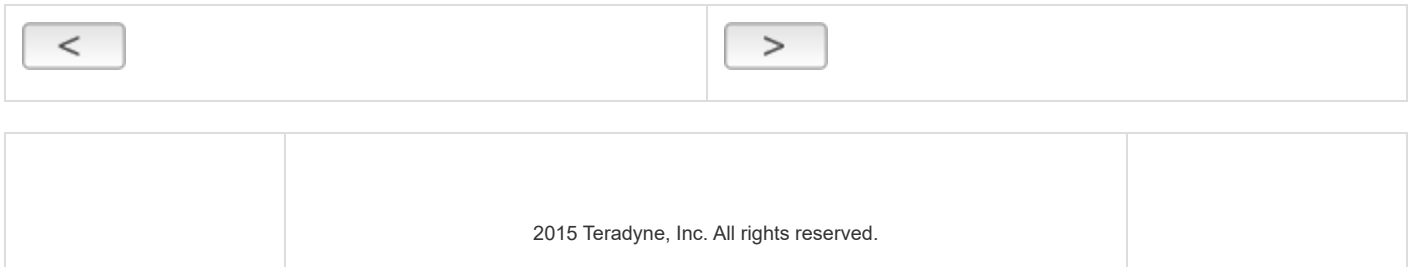
It is therefore better not to maximize the tool window. If the tool window is not maximized, TDE has scrollbars that let you see the rest of the tool window even if the TDE window is smaller than the tool window.

If the tool window is maximized, use the Restore control on the right side of the TDE toolbar, to the right of the Teradyne logo:



Restore Control

The Restore control is the one in the middle with the two stacked rectangles.



tdtde.1.05

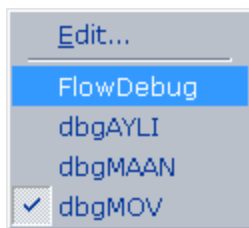


Test Debug Environment (TDE) : Using TDE : ToolSets

ToolSets

Once you arrange the TDE tools and windows as you want them, you can save this configuration so that you can restore it in a later session, thereby saving any setup time. TDE lets you save up to four named configurations, called toolsets. When you start a session, you can call up a previously defined toolset. Within a single session, you can switch between toolsets.

The TDE toolbar contains a control with a drop-down list of all available toolsets:



ToolSet Control

Selecting a toolset from the list invokes that toolset, opening all the tools defined for the toolset. During a session, you can use the menu to switch between toolsets. The toolgroup menu is located next to the Flow Domain control.

There is a built-in toolset called FlowDebug, which contains the MiniFlow, Watch, and Flow Call Stack tools. These tools are useful in combination when debugging the flow.

You can also create and modify toolsets.

To create a toolset:

1. Set up the windows as you want them.
2. Select TDE>ToolSets>Edit. The dialog box contains four buttons with a space for a name beside each.

3.

Click a button and enter a name for the toolset.
4.

Click OK.

To modify a toolset:

1.

Make the changes you want to the toolgroup window configuration.
2.

Select TDE>ToolSets>Edit.
3.

When the dialog box appears, click OK. The modified configuration is saved to the currently selected toolset.

You can select a different button to save the modified configuration to a new toolset or to overwrite an existing toolset.

Note that the TDE>ToolSets menu item also contains the names of all toolsets you have defined. Selecting a toolset displays it.

Toolsets are stored in a file named <username>_toolsets.tde. This file is located in the folder C:\Users\<username>\AppData\Roaming\Teradyne. Your toolset definitions are automatically loaded when you start TDE.

<

>

2015 Teradyne, Inc. All rights reserved.

tdtde.1.06

<

>

Test Debug Environment (TDE) : Using Debug Displays in TDE

Using Debug Displays in TDE

This section describes TDE features that apply to all debug displays. The topics are:

- Starting Debug Displays
- Maximized View and Scrollbars
- Fonts and Colors in Debug Displays
- Debug Display Common Controls
- Managing Sites
- Storing and Comparing Debug Display State
- Pin Groups in Debug Displays
- Using Summary Displays
- Debug Displays Refreshed in VBT Code

For information on specific debug displays, click [Related Topics](#).

Many debug displays allow you to generate VBT code. For more information, see [Producing Code from Debug Displays](#).

< >		
	2015 Teradyne, Inc. All rights reserved.	

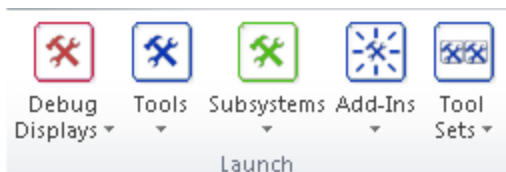
tdtde.1.07



Test Debug Environment (TDE) : Using Debug Displays in TDE : Starting Debug Displays

Starting Debug Displays

The most convenient way to open a debug display is using the Launch group on the IG-XL Tools toolbar:



Launch Group

Clicking Debug Displays displays a menu of the available instruments. Clicking Subsystems brings up a menu of the available boards.

In VBE, the IG-XL Tools toolbar is available:



VBE Controls

Clicking Instruments or Subsystems displays a menu of the available items.

Selecting an item from the menu brings up the debug display for that item. Most debug displays come up in TDE.

For more on the toolbar, see Launch Group in the DataTool section.

In addition to the buttons, you can start debug displays in these ways:

- If TDE is already up, you can use the Instruments and Subsystems commands on the TDE menu to start individual debug displays.
-  A quick way to start all relevant debug displays in TDE is the TDE>Auto Display command or button. Auto Display brings up a debug display for each instrument in a test program's current channel map. Auto Display is enabled only after you have validated the test program.

- You can start debug displays from the Tester Summary Display. Items in the Pin and Instrument columns of the display are blue links that bring up the debug display for the specified instrument set to the specified pin. See [Tester Summary Display](#).

You can start two or more instances of the same debug display. This can be useful for comparing two sites.

The lists of available instruments and subsystems are determined by your IG-XL installation and by your configuration file. If an instrument is missing from the list, verify that it is present in the configuration file.

If you start a debug display before a test program has been validated, an error appears in the display saying that the program has not been validated. If you start a display for an instrument that is in the configuration file but not in the channel map, the display shows a message that no instruments have been found.

Once a test program is validated, the display shows the current hardware settings. When you run to a breakpoint in the flow or in the VBT code, or as you step through the flow or code, the display is updated to show any changes to the settings.

As an example, you can follow these steps to start an instrument debug display.

1.

Validate a test program.
2.

Set a breakpoint on a test in the Flow Table sheet by clicking [Toggle Breakpoint](#) in the Debug group on the IG-XL Tools tab. 🖱️

The breakpoint is represented by a red dot. When you are stopped at a breakpoint, you can see valid values in the display.

3.

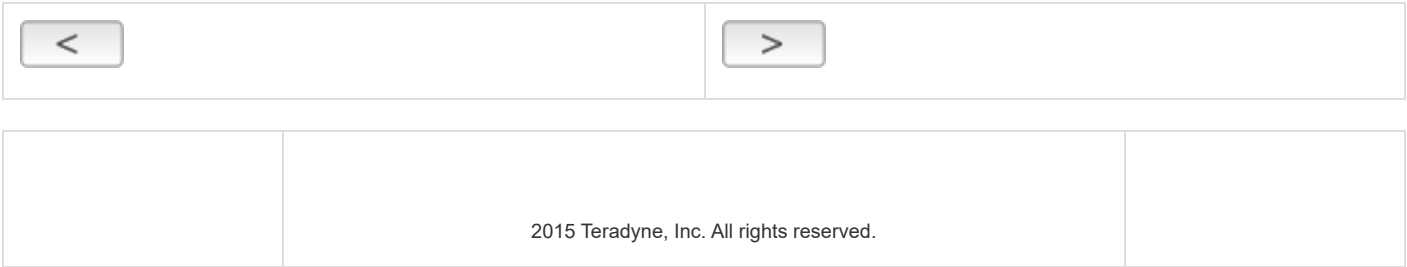
Run the test program to the breakpoint by clicking [Debug Run](#) in the Debug group on the IG-XL Tools tab. ▶️ You can step through the parts of a test instance using [Step](#). 📄 The following shows execution stopped after a test’s Body:

Flow Table						
Label	Enable	Gate			Command	
		Job	Part	Env	Opcode	Parameter
					Test	continuity
					Test	icc_standby
					Test	icc_active
					Test	checkerboard

Breakpoint in Flow Table Sheet

4. [On the](#) IG-XL Tools tab, in the Launch group, click Debug Displays.

This starts the [Test Development Environment](#) window with the instrument debug display in it. The display's settings reflect the hardware values at the time that the breakpoint is reached.



tdtde.1.08



Test Debug Environment (TDE) : Using Debug Displays in TDE : Maximized View and Scrollbars

Maximized View and Scrollbars

When a debug display first comes up in TDE, it is in a regular-sized (that is, not maximized) tool window. You can have the display expand to use the entire TDE window using the Maximize control in the upper right corner of the tool window:



Maximize Control

The Maximize control is the square in the middle. If the TDE window is smaller than the maximized display tool window, however, the TDE window has no scrollbars to let you move to another part of the debug display. This is because of an implementation constraint.

Therefore, avoid maximizing the tool window. If the tool window is not maximized, TDE has scrollbars that let you see the rest of the tool window, even if the TDE window is smaller than the display tool window.

If the tool window is maximized, use the Restore control on the right side of the TDE toolbar, to the right of the Teradyne logo:



Restore Control

The Restore control is the one in the middle with the two stacked rectangles.



	2015 Teradyne, Inc. All rights reserved.	
--	--	--

tdtde.1.09

<	>
-----------------	-----------------

Test Debug Environment (TDE) : Using Debug Displays in TDE : Fonts and Colors in Debug Displays

Fonts and Colors in Debug Displays

Debug displays use font and color changes to indicate changes to the values they display:

Black	Initial unchanged value. All numeric or string values and relay or checkbox controls are initially in black. When you click the MS button or select Store State to store the current values, all values and controls are changed back to black.
Bold	The most recently changed value or control.
Blue	A value or control you change while using the debug display.
Red (orange)	A value or control changed by the test program. This happens when you are stepping through a program with the display open.
Pink	Invalid value. If you enter an invalid value, the entire field turns pink when you exit from the field. The value itself remains blue, because an invalid value cannot be applied to the hardware. In addition to the field color, a red exclamation point appears on the toolbar with a message indicating that valid range of values for the field.
Yellow	Different values. When you select a pin group in a debug display, yellow means that the pins in the group have different values for the parameter. The value displayed is for the first pin in the group. See Pin Groups in Debug Displays.
Magenta/Green	Related to compared states. See Storing and Comparing Debug Display State.

If you place the pointer on a field, the tooltip displays the valid values for that field.

<		>	
	2015 Teradyne, Inc. All rights reserved.		

tdtde.1.10



Test Debug Environment (TDE) : Using Debug Displays in TDE : Debug Display Common Controls

Debug Display Common Controls

In addition to relevant buttons on the TDE toolbar, each debug display window's toolbar contains the following unlabeled controls:

	(Site) Specifies the site to display. Master indicates the site specified on the TDE toolbar. See Managing Sites.
	(Apply All Sites) Toggles between applying changes to all sites or only to the currently selected site. See Managing Sites. On the TDE toolbar, this command is <code>instrumentname>All Sites</code>.
	(Summary Display) Toggles between the regular schematic display and the Summary Display, which displays information on all pins. See Using Summary Displays. On the TDE toolbar, this command is <code>View>Summary</code>.
	(Copy Generated Code) Copies VBT code for the current state of the display. See Producing Code from Debug Displays. On the TDE toolbar, this command is <code>Edit>Copy Code</code>. The keyboard equivalent is <code>Ctrl+Shift+C</code>.

In addition to these controls, the command `Edit>Copy All` (keyboard equivalent `Ctrl+C`) copies the current settings of the debug display to the system clipboard. The copied ASCII text has a row of

setting names and one row per pin of setting values.

<		>	
	2015 Teradyne, Inc. All rights reserved.		

tdtde.1.11

<	>
-----------------	-----------------

Test Debug Environment (TDE) : Using Debug Displays in TDE : Managing Sites

Managing Sites

Site: 0

☒ Master

A debug display can show information for one site at a time. The Site: control specifies the site to display. Its drop-down list contains all sites. You can select a site from the list or type in a site number. You can check the Master checkbox to use the site that is defined as the Master Site in the TDE frame. See [Toolbar Controls](#).

 Any changes made to the debug display values are immediately applied to the hardware. You can use the Apply All Sites button to specify whether to apply changes only to the currently selected site or to all sites. When the button is selected, changes apply to all sites; when it is not selected, changes apply only to the currently selected site.

To compare the hardware state at two sites, you can bring up two instances of the same debug display, setting each to a different site.

<	>	
	2015 Teradyne, Inc. All rights reserved.	

tdtde.1.12



Test Debug Environment (TDE) : Using Debug Displays in TDE : Storing and Comparing Debug Display State

Storing and Comparing Debug Display State

The Store State and Compare State commands, available on the TDE menu and as TDE toolbar buttons, let you store and compare the states of the debug displays.

Store State		Stores the current state of the debug displays. When the displays first come up, a Store State is automatically executed.
Compare States		Toggles between regular mode and compare mode, which compares the currently displayed hardware values to the most recently stored values and highlights the differences.

The definitions of regular mode and compare mode are as follows:

regular mode	(Compare States button not depressed) The values in the displays represent actual hardware values. This is the standard mode for viewing and editing in TDE. See Fonts and Colors in Debug Displays .
compare mode	(Compare States button depressed). The current hardware values are compared to the most recently stored values: If a value is displayed in magenta, there is a difference between the stored value and the displayed value.

- If a value is displayed in green, there is no difference between the stored value and the displayed value.

If you edit a value while you are in compare mode, the displays return to regular mode. In regular mode, all changed values are displayed in bold. Store State and Compare State essentially let you set “checkpoints,” so you can see what has changed since the last time you executed the Store State command.

Storing and comparing states is useful in debugging test problems. For example, suppose that two tests or two devices should be identical but are giving different results. You can bring up debug displays, run the test, store the state, run the test again, and then compare the states. Any values displayed in magenta indicate a difference between the first and second runs and suggest where debugging should be directed.

You can use Store State and Compare States to generate incremental VBT code from a debug display. See [Absolute and Incremental Pins Language](#).

<		>	
	2015 Teradyne, Inc. All rights reserved.		

tdtde.1.13

<

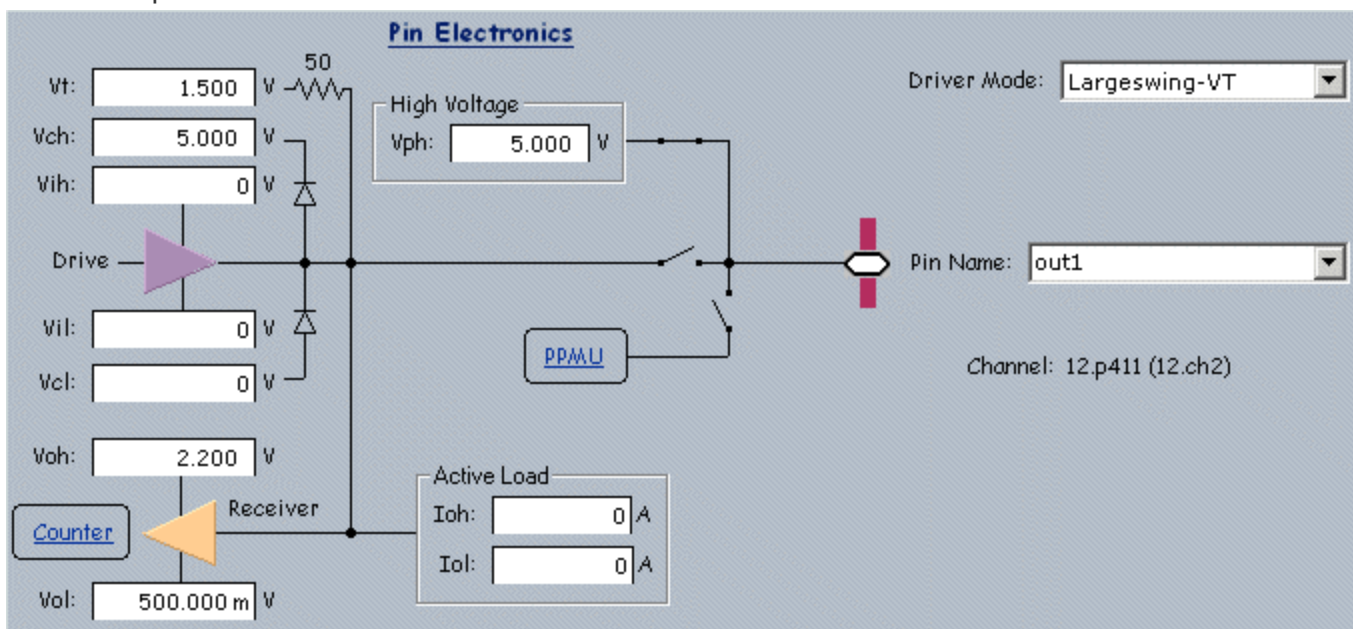
>

Test Debug Environment (TDE) : Using Debug Displays in TDE : Pin Groups in Debug Displays

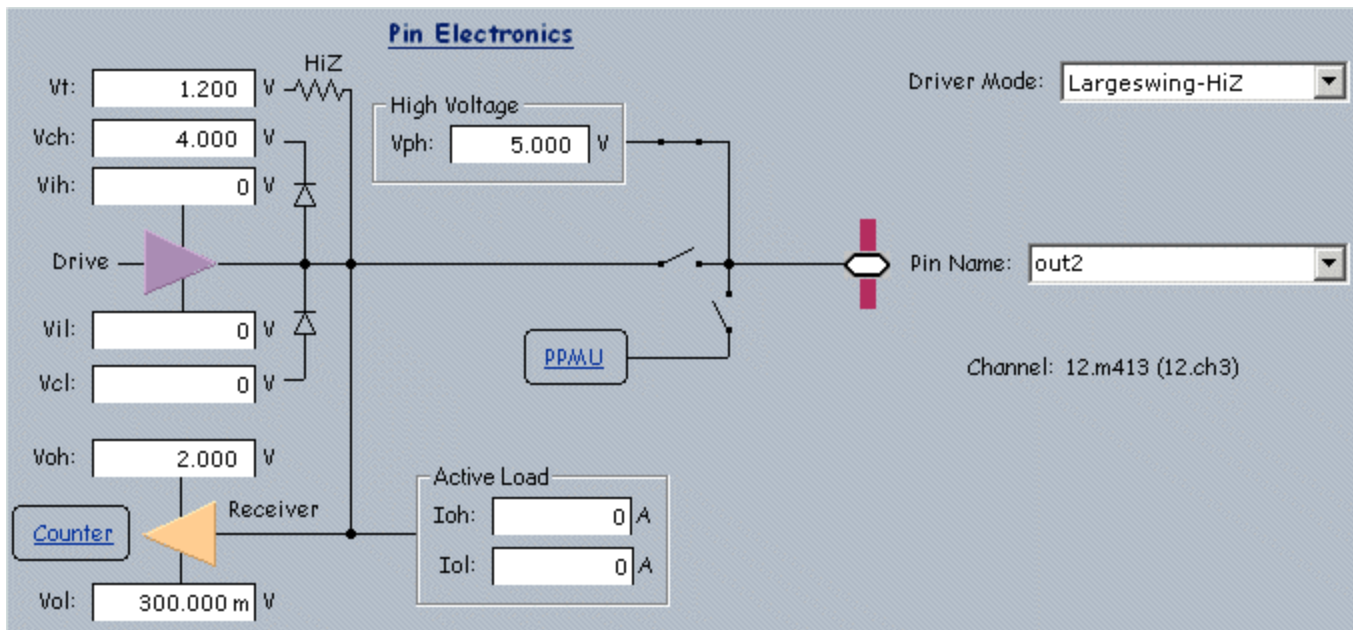
Pin Groups in Debug Displays

Debug displays include a Pin Name: control for selecting the pin to display. The control lists all pins and pin groups in the pin map. If you select a pin group, a parameter's values may not be the same for all pins. Be aware of this when you are viewing or changing values.

For example, assume that a pin group output_pins contains out1 and out2. The displays for the individual pins are as follows:

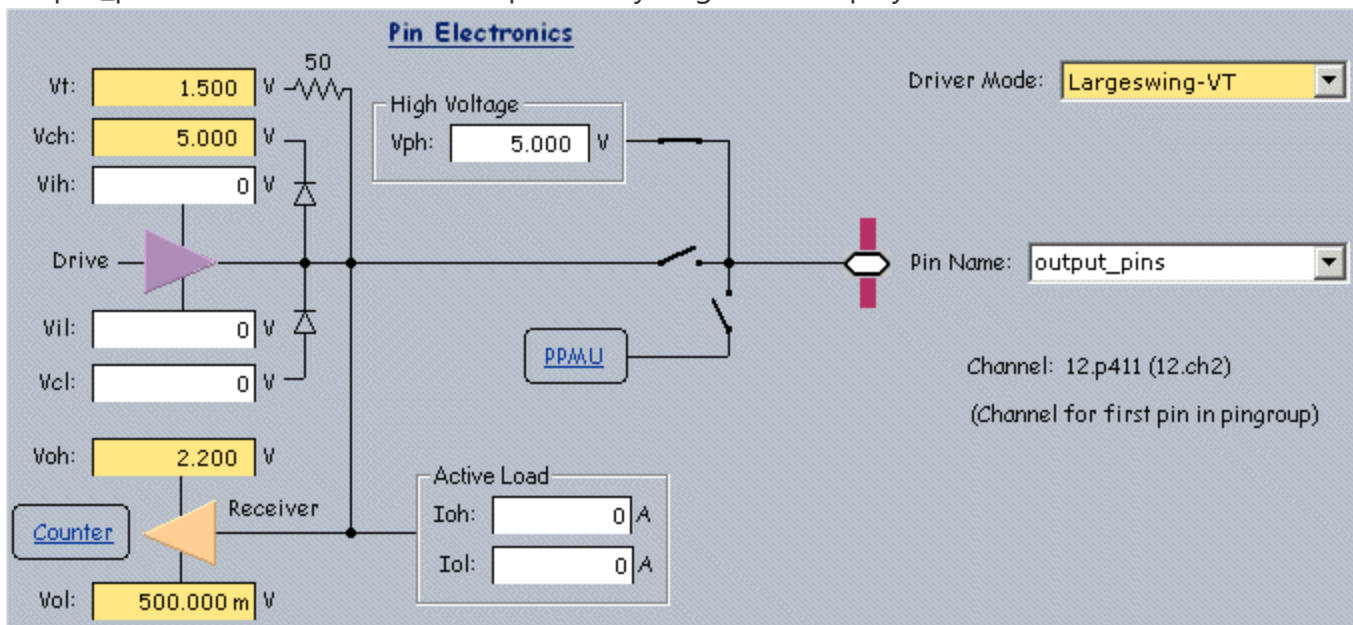


Example Debug Display for out1 Pin in output_pins Group



Example Debug Display for out2 Pin in output_pins Group

The pins have different values for Vt, Vch, Voh, Vol, and Driver Mode. If you select the pin group output_pins from the Pin name: drop-down, you get this display:



Example Debug Display for output_pins Group

The values displayed are for the first pin in the group. The yellow indicates that the parameter's values are not the same for all pins in the group.

To assign a single value to all pins for one of these parameters, type in the value (even if it is the currently displayed value) and press Enter. The box turns white to indicate that all pins now have the same value. Parameters with yellow boxes still have different values for the pins.



	2015 Teradyne, Inc. All rights reserved.	
--	--	--

tdtde.1.14

<	>
-----------------	-----------------

Test Debug Environment (TDE) : Using Debug Displays in TDE : Using Summary Displays

Using Summary Displays

Debug displays typically show a great deal of information about a single pin, arranged in a schematic diagram. At times, you may want to display a smaller amount of information about all available pins, to compare pins in a single view.

 For this purpose, a debug display may have a Summary Display. This button on the debug display's title bar toggles between the regular schematic debug display and its Summary Display. A Summary Display presents its information in tabular format. The columns are as follows:

- The first column contains a plus (+) or minus (-). Clicking this column toggles between displaying only the active site (as specified by the debug display's Active Site control) and displaying all sites.
- The second column is Pin. Each pin entry is a link. Clicking the link takes you back to the debug display, displaying information on this particular pin.
- The next columns are gray, indicating that they are read-only. These columns can include Site and Conn, which indicates whether the instrument is connected. For most of these columns, you can change the value by going back to the debug display. For example, closing a relay changes the Conn value from Open to Y or DA (if the connection is through DIB Access).
- The remaining columns are white, indicating that you can edit them. As with the debug display, any change is immediately applied to the hardware and is visible when you return to the debug display. The column may have a menu, if it permits a limited set of values; a check box, if the debug display also uses a check box; or a editable box, for a value to be entered.

To make the summary compact, some column names may be abbreviated.

A Summary Display may have radio buttons that control what subset of information is to be displayed. For example, radio buttons on the Digital Channel display let you switch between Levels information and Format/Timing information.

To view a summary of all debug displays, use the Tester Display Summary. See [Tester Summary Display](#).

< >		
	2015 Teradyne, Inc. All rights reserved.	

tdtde.1.15

<	>
-----------------	-----------------

Test Debug Environment (TDE) : Using Debug Displays in TDE : Debug Displays Refreshed in VBT Code

Debug Displays Refreshed in VBT Code

When you step through VBT code, debug displays automatically refresh. The refresh occurs anytime a language statement containing TheHdw is executed and on some statements containing TheExec. Note that if you use a With statement such as With TheHdw, you will not get a refresh on each line within the With statement.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

tdtde.1.16

	
---	---

Test Debug Environment (TDE) : Producing Code from Debug Displays

Producing Code from Debug Displays

The debug displays can generate VBT code based on the current state of the display. (This feature is sometimes referred to as “code-producing debug displays” or CPDD.)

It is sometimes more intuitive to set up an instrument using a debug display than using VBT language. Using this capability, you can run to a breakpoint and use the debug display to set up the instrument in a desired state. You can then have the debug display generate the VBT code that will put the instrument into this state. This can be either Pins language (VBT code that directly affects the instrument) or PSet language. You can then copy the VBT code into a program so that you can programmatically arrive at the same state as with the debug display.

To help in generating and using the VBT code, TDE includes the Code Viewer tool. The Code Viewer displays the most recently generated code, as well as a history of all code generated in the current session. You can copy the code from the Code Viewer into a program, edit the code in the Code Viewer, or save the code to a file. The generated code is always copied to the system clipboard, so you need not use the Code Viewer if you want to paste the code directly into a program. But the Code Viewer is useful for viewing, editing, and saving the code. For details, see Code Viewer.

The following topics are included:

- [Steps for Producing Code](#)
- [Absolute and Incremental Pins Language](#)
- [PSet Language and New PSet](#)
- [Notes on Generated Code](#)

< >		
	2015 Teradyne, Inc. All rights reserved.	

tdtde.1.17

	
---	---

Test Debug Environment (TDE) : Producing Code from Debug Displays : Steps for Producing Code

Steps for Producing Code

The following are the general steps for using a debug display to produce code:

1. Set a breakpoint in the program and run to the breakpoint.
2. If you want to use the Code Viewer to view, edit, or save code, bring it up now. On the TDE menu bar, use TDE>Tools>Code Viewer.
3. Specify whether you want the generated code to use the VB With structure. This option is on the TDE menu bar under the instrument name: for example, PPMU>Code Settings>With Block. See the note on the "With Block" Option.
4. Specify the kind of code you want to produce. The exact steps depend on the kind of code you want to produce. The options are:
 - Absolute and Incremental Pins Language
 - PSet Language and New PSet
5. Put the debug display into the state that you want to capture.
6. Copy the code using any of the following:

-  The button on the debug display
- The TDE menu bar command Edit>Copy Code
- The keyboard equivalent Ctrl+Shift+C.
7.

You can continue to make changes to the debug display and copy the resulting code. In the Code Viewer, the History View will contain all code generated during the session.

The code is copied to the system clipboard. Even if you do not have the Code Viewer open, you can paste the code from the system clipboard directly into the Visual Basic editor.

If you do have the Code Viewer open, the code is copied there as well as to the system clipboard.

For details, see [Code Viewer](#).





2015 Teradyne, Inc. All rights reserved.

tdtde.1.18

<

>

Test Debug Environment (TDE) : Producing Code from Debug Displays : Absolute and Incremental Pins Language

Absolute and Incremental Pins Language

A debug display can produce Pins language code. This is the VBT code that will set the instrument state as currently displayed in the debug display. The options for Pins language are:

Absolute Pins language	Generated code that represents the current state of the instrument based on all parameters that can set that state from within a debug display.
Incremental Pins language	Generated code that represents only debug display parameters that have been modified since a certain point.

The difference can be summarized this way:

- Absolute Pins language shows how to set a certain instrument state. It contains VBT code for all parameters.
- Incremental Pins language show how to get an instrument from one state to another. It contains VBT code only for parameters that have been modified to get the instrument from one state to the other.

The following is an example of absolute Pins language:

```
TheHdw.PPMU.Pins("digchan").ForceV 0,0.002
TheHdw.PPMU.Pins("digchan").Gate = tlOn
TheHdw.PPMU.Pins("digchan").Connect
```

TheHdw.PPMU.SetClampsVHi 8.18

TheHdw.PPMU.SetClampsVLo -3.18

The following is incremental Pins language for this example after changing one parameter:

TheHdw.PPMU.SetClampsVHi 7.5

You can use Incremental Pins language as part of debugging. While at a trap, you can manipulate a debug display to get the instrument into a state where the test is now passing. Incremental code generation lets you store the code that causes a test to pass.

Incremental Pins language uses two TDE functions:

Store State	Stores the current state of the debug display. You then make modifications to the instrument, using either the debug display or VBT Pins language.
Compare States	Compares the stored state and the current state, and highlights the changed values in the debug display.

Incremental Pins language consists of the VBT statements needed to make those changes. For more information, see [Storing and Comparing Debug Display State](#).

Functionally, the difference is this:

- Incremental Pins language is generated if you click [Copy Code](#) when TDE is in Compare States mode, that is, when the Compare States button is depressed:



Compare States Mode

- Absolute Pins language is generated if you click [Copy Code](#) when TDE is not in Compare States mode, that is, when the Compare States button is not depressed.

Producing Absolute Pins Language

[Refer to Steps for Producing Code](#). For step 4, with the debug display as the active tool, go to the TDE menu bar and select `instrumentname>Code Settings>Pins Language`. Then, when you click [Copy Code](#), the code produced will contain all the VBT Pins language necessary to put the instrument into the state shown on the debug display.

Producing Incremental Pins Language

[Refer to Steps for Producing Code](#). These instructions are for step 4, setting up the debug display. The steps for generating incremental Pins language are as follows:

1.

With the debug display as the active tool, go to the TDE menu bar and select instrumentname>Code Settings>Pins Language.

2.

At the point where you want to begin the incremental code, store the current state of the debug display. When you first bring up a debug display, an implicit Store State is executed.

MS

You can use this button or the command TDE>Store State.

3.

Make the desired changes to the instrument, using the debug display or VBT. For example, if you are debugging, you may want to get the test to be passing.

4.

At the point where you want to generate the incremental code, enable Compare States mode.

M-

You can use this button or the command TDE>Compare States.

5.

Then, when you click Copy Code, the code produced will contain the VBT Pins language necessary to get the instrument from the original state to the new state.

If no changes are made to the instrument after the initial state, there is no incremental code to be generated. Clicking Copy Code in this case causes an error message to be displayed in the debug display.

Note that generated incremental code includes only state changes, that is, changes to parameter values. Actions, such as meter reads or triggers are not recorded and don't generate code.

tdtde.1.19



Test Debug Environment (TDE) : Producing Code from Debug Displays : PSet Language and New PSet

PSet Language and New PSet

You can set some instrument parameters using parameter sets, or PSets. A debug display can produce code that specifies these parameters as PSet language, as opposed to Pins language. Because only some parameters can be set using a PSet, code produced by PSet options contains a mixture of Pins language and PSet language. Any parameters that cannot be set using a PSet are represented by Pins language. The rest use PSet language. For example, the following code example (shown with some parameters deleted for space) has Pins language first for the parameters that cannot be set by a PSet, and then the PSet parameters:

```
'With TheHdw.DCVI.Pins("p21")
'.VoltageRange.Autorange = True
...
'.NominalBandwidth.Value = 5000
'.Meter.Filter.Value = 10000
'.Meter.Filter.Bypass = False
...
'.Alarm(tlDCVIArmOpenLoop) = tlAlarmForceFail
'.Alarm(tlDCVIArmMode) = tlAlarmForceFail
...
'.Disconnect tlDCVIArmConnectHighForce
'.Disconnect tlDCVIArmConnectHighSense
'With .PSets("pset_rdson")
'.Mode = tlDCVIModeCurrent
'.ComplianceRange(tlDCVIArmCompliancePositive) = 5
'.ComplianceRange(tlDCVIArmComplianceNegative) = 0
```

```

'.VoltageRange = 5
'.CurrentRange = 4
'.Voltage = 5
'.Current = 0.1
'.Meter.Mode = tIDCVIMeterVoltage
'.Meter.VoltageRange = 5
'.Source.SampleRate = 0
'.Source.StopOnModeAlarm = False

```

```
'End With
```

```
'End With
```

Note that the code in these examples has the With Block option enabled. If it is not enabled, there is a separate statement for each parameter in both Pins language and PSet language. For more information, see ["With Block" Option](#).

The PSet options under instrumentname>Code Settings are:

New PSet	Generates the code for a new PSet.
PSet Language	Generates the code for the currently applied PSet, as defined below.

New PSet generates PSet statements for the parameters and also generates a PSets.Add statement before the Pins language and PSet language statements.

For example:

```

TheHdw.DCTime.Pins("p10").PSets.Add "pset1"
'With TheHdw.DCTime.Pins("p10")
'.Mode = tIDCTimeModeOff
...
'With .PSets("pset1")
'.FrontEnd.VoltageRange.SetValue 0, tIDCTimeImpedanceNone
'.FrontEnd.Threshold1 = 0
'.FrontEnd.Threshold2 = 0
'.FrontEnd.Hysteresis = tIDCTimeHysteresisOff
'.Stamper1.Event.Input = tIDCTimeInputNone
...
'End With
'End With

```

The Add and PSet statements create the placeholder name "pset1". You can change this name as appropriate.

PSet Language does not create a placeholder name or include an Add statement. Instead, it uses the name of the currently applied PSet.

For example:

```
'With TheHdw.DCVI.Pins("p21")
...
'With .PSets("pset_rdson")
...
'End With
'End With
```

The currently applied PSet is the PSet that you have selected and applied using the PSet control on the debug display. If you have not explicitly applied a PSet using this control, there is no currently applied PSet, and the string "REPLACE_WITH_REAL_PSET" is used instead:

```
'With .PSets("REPLACE_WITH_REAL_PSET")
```

Some instruments do have VBT statements that set each parameter separately. In such a case, the generated code uses the instrument's PSet().Set command, which sets all parameters in a single statement.

For more information, see [Parameter Sets \(PSets\)](#) in the Programming Concepts section.

<

>

2015 Teradyne, Inc. All rights reserved.

tdtde.1.20

<	>
-----------------	-----------------

Test Debug Environment (TDE) : Producing Code from Debug Displays : Notes on Generated Code

Notes on Generated Code

This section describes the code that is generated from debug displays. The following topics are available:

- ["With Block" Option](#)
- [Signals](#)
- [Alarms in Pins Language](#)
- [Multiple Pins](#)
- [Displays with Tabs](#)
- [Variables in Code](#)

"With Block" Option

By default, code produced from a debug display does not use VB's With block structure. Instead, the code has a separate full VBT statement for each parameter that is set. You can specify that you want the code to be produced in With block format. With the debug display as the active tool, go to the TDE menu bar and select instrumentname>Code Settings>With Block. If you select this option, the code produced is completely commented out, with a warning comment at the beginning. To use the code in a program, you will need to uncomment it.

The option is made available in this way because of a VBA restriction. If you try to insert a With block or End With into a function while it is running and you are stopped at a breakpoint, VBA will require you to reset the project. Specifically, the restriction applies to any subroutine or function currently on the call stack. You can insert the With block into a function that is not on the call stack. Also, if you have a pre-existing With block in your running function, you can insert regular statements (such as `.Voltage = 3`) into it.

Because of this restriction, the default is not to use the With block structure. If you select the With block option, the code is commented out so you can insert it into a running function without requiring a reset, and then uncomment it later; or you can insert it, uncomment it immediately, and accept the reset. The With block coding style is made available as an option, because it is cleaner and better performing code than separate VBT statements.

If only one line of code is to be generated (for example, in incremental mode), the With statement is not used, even if it is enabled.

Signals

If a debug display is for a signal-based instrument, it generates Pins language for signal-related parameters by including a `.Signals` node with a dummy signal name.

For example:

```
'With TheHdw.VHFACSource.Pins("srcpos")
'.Signals("REPLACE_WITH_REAL_SIGNAL").Amplitude = 1
'.SampleRate.Enable = False
'.Signals("REPLACE_WITH_REAL_SIGNAL").SampleRate = 99999999.8137355
'.Signals("REPLACE_WITH_REAL_SIGNAL").LowPass.Bypass = True
...
'.Signals("REPLACE_WITH_REAL_SIGNAL").Coupling = tIVHFACSourceCouplingDC
'.Condition.Connect(tIVHFACSourceConditionToBoth)
...
'.Signals("REPLACE_WITH_REAL_SIGNAL").CommonMode.DC = 0
'.Signals("REPLACE_WITH_REAL_SIGNAL").OutputDelay = 0
'.Signals("REPLACE_WITH_REAL_SIGNAL").LoadSettings
...
'End With
```

The string `"REPLACE_WITH_REAL_SIGNAL"` is used instead of a signal name. If any Signals code is generated, the code also includes a `.LoadSettings` command.

Signals parameters that are not represented on the debug display do not have code generated. You must create your own WaveDefs and Signals for this Signals language to function.

Alarms in Pins Language

An instrument's debug display includes an Alarm box that lets you specify the behavior for each alarm. This alarm information is included as part of the Pins language:

- For absolute language, the code includes VBT statements setting all alarms to their currently specified behavior.
- For incremental language, only the alarm currently displayed in the Alarm box is included in the code. Code is not produced for any other alarm actions that might have taken place during the session.

Note that code is not generated for the instrument's tAlarmAll option. Code is generated only for individually specified alarms.

Multiple Pins

A debug display may contain setting for multiple pins: for example, a Pos and Neg pin on a differential instrument. In such cases, code is generated for only one pin. This is the most recently selected pin; if no pin has been selected, the first pin encountered is used.

If some debug displays parameters are pin-specific, the generated code specifies the pin for which the parameter is set. If the With Block option is set, each pin has its own With block. For example:

```
With TheHdw.BBACSource.Pins("cap1pos")
```

```
    .VoltageRange.Autorange = True
```

```
    .Amplitude = 1
```

```
    .VoltageRange = 1.28
```

```
    ...
```

```
    .MicroCodeControl = tBBACSourceMicroCodeControlByCPU
```

```
End With
```

```
With TheHdw.BBACSource.Pins("cap1pos")
```

```
    .Disconnect tBBACSourceFromPOS,tBBACSourceToCCC
```

```
    .Disconnect tBBACSourceFromPOS,tBBACSourceToDA
```

```
    .Disconnect tBBACSourceFromPOS DA,tBBACSourceToDUT
```

```
    ...
```

```
End With
```

```
With TheHdw.BBACSource.Pins("src1ref")
```

```
    .Disconnect tBBACSourceFromREF,tBBACSourceToCCC
```

```
    .Disconnect tBBACSourceFromREF,tBBACSourceToDA
```

```
    .Disconnect tBBACSourceFromREF DA,tBBACSourceToDUT
```

```
    ...
```

```
End With
```

Displays with Tabs

Some debug displays have tabs. For example, there may be a Source tab for controls relating to sourcing and a Capture tab for controls related to capturing. Only one tab can be displayed at a

time.

If a debug display has tabs, code is generated only for the controls on the currently displayed tab or tabs. For such displays, to get complete code that includes all controls, you must expose the additional tabs and generate more code, and then edit the results if necessary.

Variables in Code

In general, the generated code does not use variables. Instead, the code uses the hardcoded values (or their language equivalents) that are seen on the debug display.

Some debug displays, however, have structures of data for parameters values. For these, variables are declared, assigned values, and used in the code that is generated. No attempt is made to prevent creation of duplicate variable names.

< >		
	2015 Teradyne, Inc. All rights reserved.	

tdtde.1.21



Test Debug Environment (TDE) : Debugging in TDE

Debugging in TDE

TDE provides a convenient, integrated environment for debugging test programs.

The major components of the debugging environment are as follows:

- TDE has a set of debug buttons and commands, consistent with those found in DataTool and in the VB environment, that you can use for stepping through the flow.

See Debug Menu.

- The MiniFlow tool provides a convenient subset of the Flow Table sheet. You can step through the tests in the flow using the TDE debug buttons and commands. You can also step into the VBT code for a test instance.

See Debugging Using MiniFlow.

- A set of TDE tools can be used with MiniFlow:
 - The Results Tool provides a convenient subset of datalog output. As you step through the tests in MiniFlow, Results displays the results of the most recently executed step.
 - MiniBin comes up when you step into a Bintable opcode in MiniFlow. MiniBin lets you step through the logic of the bin table as defined on the Bin Tables sheet.
 - Watch lets you watch how the pass/fail result, the bin number, and the sort number are set as you step through MiniFlow and MiniBin.

–

FlowCallStack lets you view the call stack of subflows.

- When you step into a VBT test from MiniFlow, the VBT Debugger is invoked (if you have enabled it). The VBT Debugger provides a subset of the functionality of the Visual Basic Editor (VBE) and offers an integrated debugging environment with the rest of the debugging tools.

See VBT Debugger.

- Debug displays are available for each instrument. A debug display gives a schematic view of the instrument for each pin and lets you view and change hardware settings. A Summary view gives a tabular display of a subset of the information, but for all pins. TDE lets you store the current state of a debug display and then compare a later state to find differences.

Each debug display is described with its instrument. For more information, see Using Debug Displays in TDE.

- PatternTool lets you create, view, edit, and debug patterns. It also lets you view HRAM from pattern execution.

See About PatternTool in the PatternTool section.

<

>

2015 Teradyne, Inc. All rights reserved.

tdtde.1.22

	
---	---

Test Debug Environment (TDE) : Debugging in TDE : Debug Runs in DataTool and TDE

Debug Runs in DataTool and TDE

IG-XL and TDE support several debugging scenarios. Depending on the scenario you choose and whether DataTool or TDE is maximized, you will see different window behavior, as follows:

- If you do debug runs or debug steps in DataTool, TDE is brought up and minimized at the start of the very first debug run. The DataTool Flow Table shows the execution pointer. If you want to use debug displays or other TDE tools, you can select TDE in the taskbar and then open the tools.
- If you do debug runs or debug steps in TDE while you have the MiniFlow tool open, you will see the execution pointer in MiniFlow. See Debugging Using MiniFlow.
- If you do debug runs or debug steps in TDE but you do not have the MiniFlow tool open, DataTool is brought forward to display the execution pointer. If the DataTool window is in the maximized state, TDE will not be visible, but is still available for selection from the taskbar.

	
---	---

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

tdtde.1.23



Test Debug Environment (TDE) : Standard TDE Commands and Buttons

Standard TDE Commands and Buttons

TDE has a standard set of command menus and corresponding toolbar buttons. When a tool is active in TDE, its menus are merged with TDE's, so both the standard TDE commands and the tool-specific commands are available. In addition, standard TDE toolbar buttons and other toolbar controls are also always available, no matter what tool is currently active.

The categories of standard TDE commands and buttons are:

- File Menu
- Edit and View Menus
- TDE Menu
- Debug Menu
- Window Menu
- Toolbar Controls

For tool-specific commands and buttons, see the section on the specific tool.

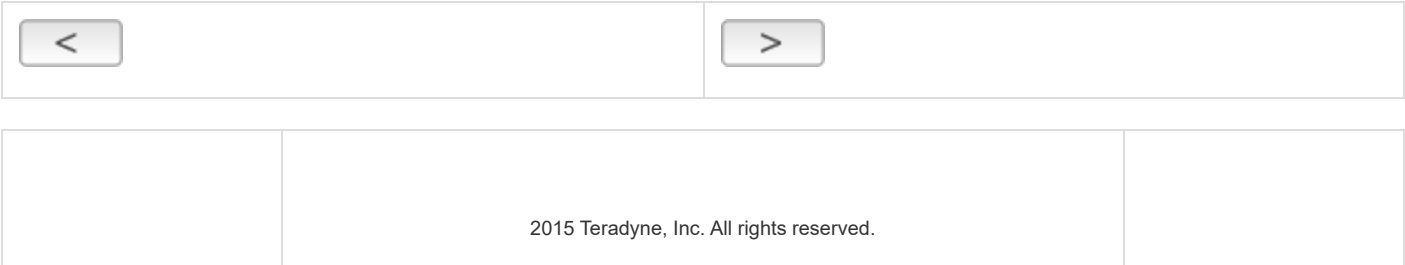
Note on Menus and Toolbars

You can only select one tool at a time. Specifically, only one toolgroup window can be active, and only one tool in the toolgroup can be active. To select a specific tool and make it active, click its icon in the toolbar of its toolgroup window.

When you select a tool, its toolbar is shown at the top of its toolgroup window.

In addition, when you select a tool, its menus are merged with the standard TDE menus. This means that, at any time, all the accessible menu commands and toolbar buttons are valid for the active tool. (The debug commands and buttons are a partial exception. They are visible but grayed out if not valid for the currently active tool.)

When you select a tool, the menu bar may include additional items. If the tool has editing or viewing commands, Edit or View is added to the menu bar. If the tool has functions that are specific to it, the tool's name is added to the menu bar. Tool-specific commands often have corresponding buttons on the tool's toolbar.



tdtde.1.24

<	>
-----------------	-----------------

Test Debug Environment (TDE) : Standard TDE Commands and Buttons : File Menu

File Menu

The only standard TDE command is:

Exit TDE	Closes all toolgroup windows and exits from TDE.
----------	--

Other commands on the File menu are tool-specific. One command that appears for all tools is:

Close toolname	Closes the currently active tool without exiting from TDE.
----------------	--

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

tdtde.1.25

<	>
-----------------	-----------------

Test Debug Environment (TDE) : Standard TDE Commands and Buttons : Edit and View Menus

Edit and View Menus

Most commands on the Edit and View menus are tool-specific. The following are available when a debug display is the currently active tool:

Edit>Copy All	 (Ctrl+C) Copies the current settings of the debug display to the clipboard. The copied text has one row of setting names and one row per pin of setting values. It is tab-delimited ASCII text.
Edit>Copy Code	 (Ctrl+Shift+C) Copies VBT code for the current state of the display. See Producing Code from Debug Displays.
View>Summary	 Toggles between the regular schematic display and the Summary Display, which displays information on all pins. See Using Summary Displays.

<	>
-----------------	-----------------

tdtde.1.26

	
---	---

Test Debug Environment (TDE) : Standard TDE Commands and Buttons : TDE Menu

TDE Menu

Commands on the TDE menu are independent of any specific tool and are used to manipulate the entire TDE environment. Several TDE menu commands have toolbar equivalents.

Auto Display



Brings up a debug display for every instrument in the current channel map. Auto Display is enabled only after the test program has been validated.

Restore Last Session



Restores TDE to the state it was in when you last closed TDE, with the same tools in the same position.
If tools are already open in TDE, this command adds any additional tools that were in the previous session.
TDE stores its state in the PrevState.tde file in the folder containing the test program. Not every change to the tools displayed is stored as state. For example, if a tool is brought up but is not used for debugging, it is not restored. This function tries to recreate the previous debugging session.

ToolSets

Has one command, Edit, that brings up the dialog box for saving the current configuration of tools as a toolset.
The menu also contains the names of all toolsets you have defined, plus the built-in toolset FlowDebug. Select a name to bring up that toolset. See ToolSets.

Tools	Opens the selected tool. Some tools can only be opened once. For example, only one MiniFlow can be open, because multiple MiniFlows would be redundant (unlike a debug display, which can provide multiple meaningful views of the hardware). For such tools, if one instance is already open, its name is grayed out in the list of available tools.
Instruments	Opens the debug display for the specified instrument.
Subsystems	Opens the debug display for the specified subsystem.
Store State	 Stores the current state of the debug displays. When the displays first come up, a Store State is automatically executed. See Storing and Comparing Debug Display State.
Compare State	 Toggles between compare mode and regular mode. See Storing and Comparing Debug Display State.
Validate Job	 Checks the current job for validity. This is required before execution. See Validating a Test Program in the DataTool section.
Characterization Editor	 Brings up Characterization Studio. See Characterization Studio.
Excel	 Brings the Excel workbook to the front, on the currently selected DataTool worksheet. To return from DataTool to TDE, on the IG-XL Tools tab, in the One-Click Launch group, click Test Debug Environment.
Visual Basic Editor	 (Alt+F11) Brings up the Visual Basic for Applications editor. Note that the Visual Basic editor is also brought up if you step into a test instance based on a VBT test (from MiniFlow).
Refresh	(F12) Refreshes the current display.
Options	Lets you set options for using TDE. Currently the only option is Use VBT Debugger. This command toggles between enabling and disabling the VBT Debugger. See VBT Debugger.

Toolbars

Toggles whether to display toolbars under the TDE menu.

<		>	
	2015 Teradyne, Inc. All rights reserved.		

tdtde.1.27

<

>

Test Debug Environment (TDE) : Standard TDE Commands and Buttons : Debug Menu

Debug Menu

Debug functions are as follows. See also [Debugging Using MiniFlow](#) in the MiniTools section.

Run	 (F5) Begins running the program or continues execution from the current stopping point to the next breakpoint or the end of the flow. See Selecting Run Options .
Stop	 Stops program execution in debug mode. See Debug Run and Debug Stop .
Step Into	 (F8) Executes the current item one line at a time by stepping into the lower-level code: for example, by stepping into a test template or test procedure from the flow. If there is no lower-level code, Step Into is equivalent to Step Over.
Step Over	 (Shift+F8) Executes the next item in the flow without stepping into the current item. See Step , Step Over , and Step Out .
Step Out	 Executes the remaining lines of the current item and then steps back up to the next higher level: for example, from a test procedure back to the flow. See Step , Step Over , and Step Out .

	Step Out is enabled only if there is a higher level to which execution can return.
Toggle Breakpoint	 Sets or removes a breakpoint on the current row. You can also set or clear a breakpoint by clicking in the MiniFlow margin to the left of the row number. See Toggle Breakpoint.
Clear Breakpoints	Removes all breakpoints in the flow. See Toggle Breakpoint .
Clear All Breakpoints	(Ctrl+Shift+F9) Removes all breakpoints in the flow and in all procedures. See Toggle Breakpoint.
Call Stack , Set Next Run to Element	These commands are part of a legacy IG-XL feature that is not recommended for new test development. See Note on Test Procedures , Test Elements , and PDE in the DataTool section.

< >		
	2015 Teradyne, Inc. All rights reserved.	

tdtde.1.28



Test Debug Environment (TDE) : Standard TDE Commands and Buttons : Window Menu

Window Menu

These commands apply to the currently selected tool or toolgroup window. They are also available when you right-click a tool's icon in the toolgroup window toolbar.

Next Tool	(F6) Displays the next tool in the current toolgroup window. The next tool is defined as the tool represented by the next icon in the window's toolbar.
Docking	Docks the current toolgroup window. The command has a pull-right menu with None, Left, Right, and Bottom. Select one of Left, Right, and Bottom to dock the toolgroup window to the specified side of the TDE window. Select None to undock the window. If you detach a tool from a toolgroup window, you can use this command to dock the newly created toolgroup window.
Full Screen	 Toggles between expanding the current window to fill the entire TDE workspace and using a smaller window.
Close 'toolname'	Closes the tool. If this is the only tool in the toolgroup window, the toolgroup window closes as well. The TDE File menu also contains a Close command that can close the currently active tool.
Detach 'toolname'	Removes the tool from this toolgroup window and moves it to a new toolgroup window by itself.
Cut 'toolname'	Removes the tool from this toolgroup window to allow you to paste it into a different toolgroup window. If this is the only tool in the

	toolgroup window, the toolgroup window closes. After cutting a tool, select an icon in the toolgroup window to which you want to paste the tool. Then right-click and select Paste Tool.
Paste Tool	Adds the cut tool to the current toolgroup window. This command is enabled only if a tool has been previously cut but has not yet been pasted.
1 <tool1>	
2 <tool2> ...	Shifts the focus to the specified tool within TDE. There is a numbered entry for every tool currently open in TDE.

For more information, see [Using Toolgroup Windows](#).

<	>	
	2015 Teradyne, Inc. All rights reserved.	

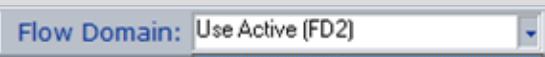
tdtde.1.29



Test Debug Environment (TDE) : Standard TDE Commands and Buttons : Toolbar Controls

Toolbar Controls

The following controls appear in the TDE toolbar:

	Master site: Defines a common site to be used by the TDE tools. For example, most debug displays show site-specific information. The Site control on each individual debug display has a Master Site option that lets you set the tool's site to whatever this Master Site control has selected. This Master site control is synchronized with the Master site control in the Tool Actions Group on the IG-XL Tools tab
	Flow Domain: Defines a flow domain pin group for the current concurrent subflow. See Using Flow Domains.
	(Tool Set) Displays the currently defined toolsets and lets you select the toolset to be invoked. See ToolSets.

<	>	
	2015 Teradyne, Inc. All rights reserved.	

tdtools.2.01

<

>

TDE Tools

TDE Tools

This section describes [TDE tools included on the TDE>Tools menu](#). These tools offer views of the DataTool sheets that relate to the current program or to results displayed by the program.

[TDE tools described in this section are:](#)

- [Code Viewer](#)
- [FlowCallStack](#)
- [MiniBin](#)
- [MiniFlow](#)
- [PSet and Signals Editor](#)
- [Results Tool](#)
- [Tester Summary Display](#)
- [VBT Debugger](#)
- [Watch](#)

The TDE>Tools menu also brings up other tools and debug displays. Each has help associated with it.

< >		
	2015 Teradyne, Inc. All rights reserved.	

tdtools.2.02

<

>

TDE Tools : Code Viewer

Code Viewer

 This is the icon for the Code Viewer.

The Code Viewer is a TDE tool that displays the code produced by debug displays. The information here is specifically about using the Code Viewer. It does not cover generating the code or the content of the generated code. For details on generating code, see Producing Code from Debug Displays.

To bring up the Code Viewer, use this command on the TDE menu bar: TDE>Tools>CodeViewer. Note that the Copy Code function on the debug display copies the generated code to both the system clipboard and to the Code Viewer. You can therefore paste code directly into VBE without using the Code Viewer. The Code Viewer, however, does let you view multiple instances of copied code, edit code, and save it to a file.

The Code Viewer has two tabs:

Current View	Shows the most recently generated code. Each time you generate code, the new code replaces whatever code was in the Current View. You can copy part or all of the code in the Current View and paste it into a test program. Note that you cannot edit or save code in the Current View.
History View	Shows all code that has been generated since the start of the session. Each time you generate code, it is appended to the History View. Each block of generated code is preceded by a comment that includes the date and time of generation, plus the instrument debug display that generated it. Several debug displays can write to the History View during a single session. You can edit and save the code in the History View.

You can generate code from debug displays when the Code Viewer is not present. If you then bring up the Code Viewer, the History View contains all code generated during this session. The Current View displays only code that has been generated while the Code Viewer is present.

The Code Viewer has the following buttons on its toolbar:

	Cut
Ctrl+X	History View only. Copies the currently selected text to the system clipboard and deletes the text from the History View. Enabled only if text is currently selected.
	Copy
Ctrl+C	Copies the currently selected text to the system clipboard. Enabled only if text is currently selected.
	Paste
Ctrl+V	History View only Pastes the text currently on the system clipboard to the current location in the History View.
	Clear
	Clears the current View.
	Save
Ctrl+S	History View only Brings up a file browser to let you specify the file. Saves the entire contents of the History View as ASCII text to a .txt file.
	Find text
Ctrl+F	History View only Finds the specified text in the History View. Once you have specified the text, you can use F3 to find the next instance.

These functions are also available on the TDE Edit menu, except for Save, which is on the TDE File menu.

tdtools.2.03



TDE Tools : FlowCallStack

FlowCallStack

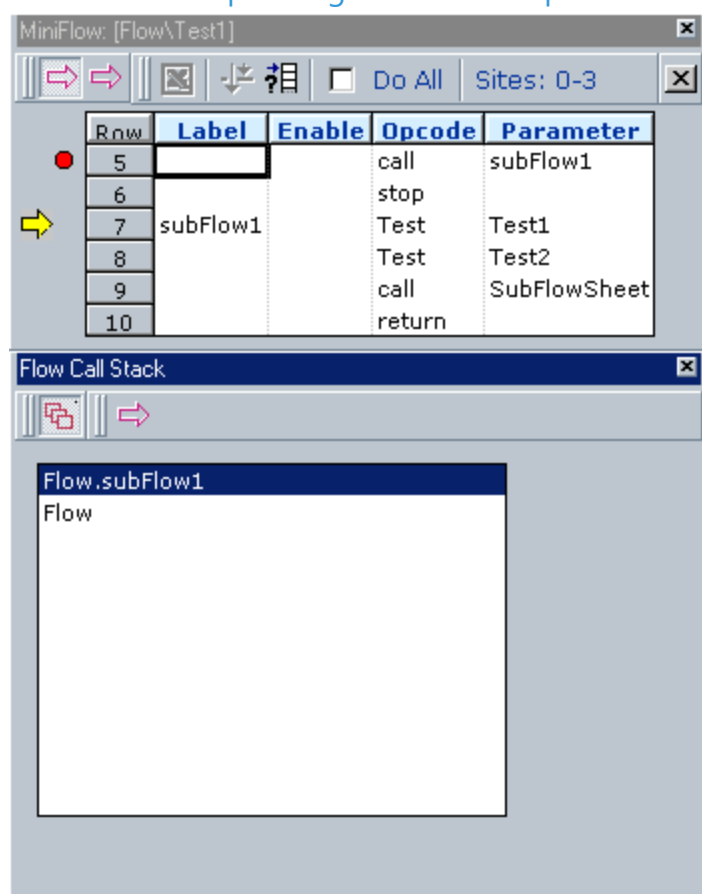


This is the icon for the FlowCallStack tool.

This tool is used for debugging a program that uses subflows. See [Subflows](#).

The built-in toolset FlowDebug includes the FlowCall Stack tool, as well as Watch and MiniFlow.

When execution is stopped at a breakpoint, the FlowCallStack tool displays an ordered list of the subflows corresponding to the call sequence of the flow step where the job is currently stopped.

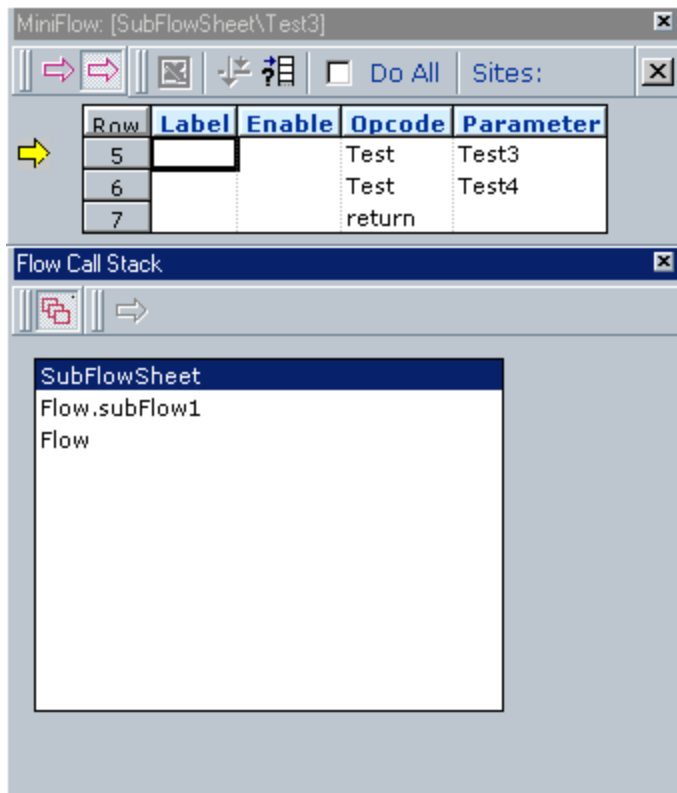


FlowCallStack Window

The current subflow is the first in the list. The main Flow Table sheet where execution started is the last in the list. If a subflow is an entire Flow Table sheet, the sheetname appears in the list. If a subflow is on the same Flow Table sheet as the main flow, it has the format mainflowsheetname.subflowlabel: for example, Flow.subFlow.

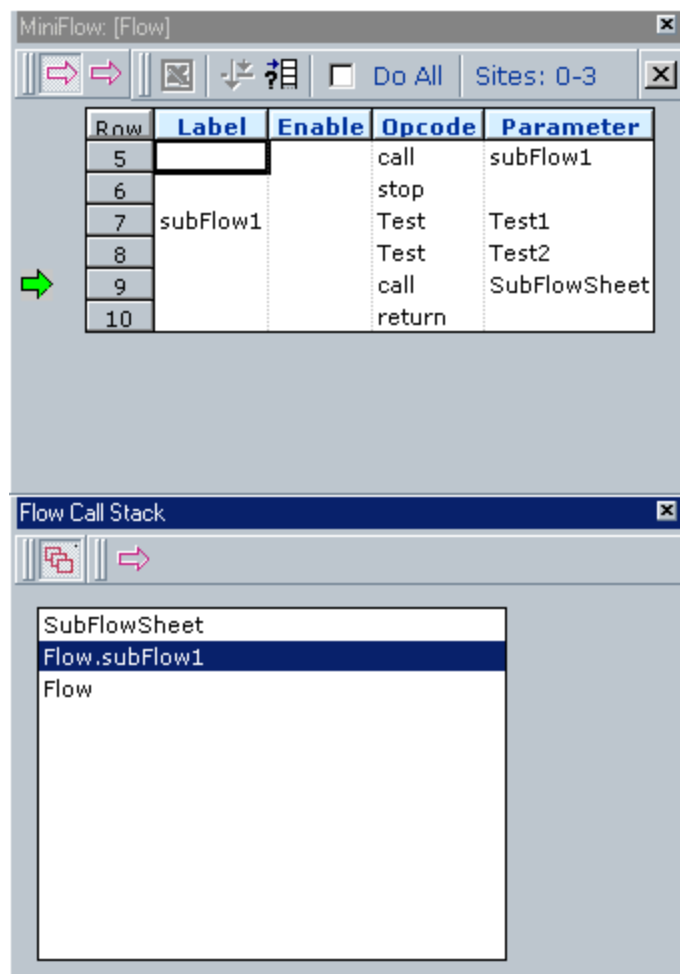
FlowCallStack is used with MiniFlow. When the main flow calls a subflow on the same sheet, the yellow execution pointer in MiniFlow indicates where execution is stopped and FlowCallStack shows the list of subflows.

When the called subflow is on a different Flow Table sheet, a new instance of MiniFlow is opened for the called sheet. The yellow pointer indicates where execution is stopped, and FlowCallStack adds the sheet name to the top of the list:



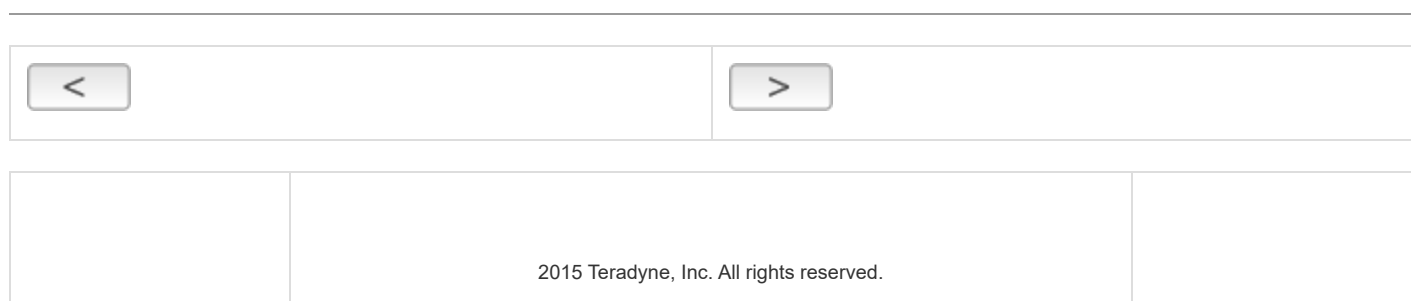
FlowCallStack Window Showing Subflow from a Different Flow Table

If you double-click any flow or subflow in FlowCallStack, the instance of MiniFlow containing the flow or subflow comes forward, with a green pointer indicating where the call to the subflow was made:



Choosing a Subflow from the FlowCallStack Window

When a MiniFlow instance with a green pointer is visible, you redisplay the MiniFlow with the yellow current execution pointer by double-clicking the flow that is at the top of the FlowControlStack.



tdtools.2.04



TDE Tools : MiniBin

MiniBin



This is the MiniBin icon.

The MiniBin tools lets you view the step-by-step execution of a bin table. See [Bin Tables](#) in the DataTool section.

To bring up MiniBin, use TDE>Tools>MiniBin. This displays the first bin table on the current Bin Tables sheet. Also, if you are stopped in MiniFlow on a Bintable step, using Step Into brings up MiniBin with the correct bin table displayed. See [Bin Tables and Subflows](#) in the section on debugging with MiniFlow.

If other bin tables are defined on the Bin Tables sheet, you can use the drop-down list on the toolbar to display them.

The yellow execution pointer indicates the current bin table step. Use the Step Into or Step Over button to move the execution to the next step. Step Out executes the rest of the bin table and steps out of MiniBin to the next step of MiniFlow.

The bin table shows the results for the currently selected site, as selected by the site control on the TDE toolbar. When a bin table row causes a site to be removed from the active site list, it is no longer displayed in the bin table.

As each step is executed, the flag states change color to indicate whether the items for the currently selected site matched the expected states:

Row	Op	Sort	Bin	Result	Test1	Test2	Test3
1	AND		1	Pass	P	P	P
2	OR		11	Fail	F	F	F
3	AND		22	Fail	F	F	F
4	AND		33	Fail	P	F	F
5	AND		44	Fail	F	P	
6	AND		44	Fail			

MiniBin Showing Test Results

Green means that the item matched the expected state. Red means that it did not match. As soon as an item does not meet the row’s condition, MiniBin stops evaluating the items, and the remaining items stay black. For example, for an AND condition, the first item that does not match the expected state means that the row’s condition cannot be met: the result of the items are not evaluated. For an OR condition, as soon as one item matches its expected state, the row’s condition is met and the rest of the items are not evaluated.

You can edit the bin table in MiniBin. Edit the Op, Result, and flag columns by selecting from the drop-down list. Edit the Sort and Bin columns by double-clicking in the cell and typing a new value. Changes to MiniBin are written immediately to the Bin Tables sheet; they take effect when the job is revalidated.

<		>	
	2015 Teradyne, Inc. All rights reserved.		

tdtools.2.05

<

>

TDE Tools : MiniFlow

MiniFlow

 This is the icon for the MiniFlow tool.

The MiniFlow tool is a subset view of the Flow Table of the currently active job. The tool is especially useful for debugging in TDE.

As Flow Table sheets become more complex, with options for calling subflows and bintables, for conditional execution of individual flow steps and blocks of flow steps, and for continuing on fail, it can become difficult to determine how a flow will execute. MiniFlow provides a simplified, customizable view of the Flow Table, as well as ways to see results at each step, especially when using the Watch tool.

The built-in toolset FlowDebug includes the MiniFlow tool, as well as Watch and FlowCall Stack. See ToolSets in the TDE section.

The following topics are included:

- Columns Displayed
- Run Options Panel
- Row Display
- Debugging Using MiniFlow
- Setting Breakpoints and Stepping Through the Flow
- Displaying Results and Sites

•	Adding and Deleting Rows and Editing Values
•	Using Enable Words
•	Step Not Executed (Reason Disabled)
•	Bin Tables and Subflows
•	Assume and Simulate
•	Using the Find and Filter Commands
•	Displaying Test Name, Number, and Limits
•	Using the Find and Filter Commands
•	Commands and Toolbar Buttons

tdtools.2.06

<

>

TDE Tools : MiniFlow : Columns Displayed

Columns Displayed

MiniFlow displays a subset of columns of the currently active DataTool Flow Table sheet. Certain columns are included by default. You can add other columns and reorder the display. Most of the columns obviously correspond to columns of the Flow Table sheet. Others provide information not found in any single Flow Table sheet column.

Default Columns

By default, these columns are displayed in MiniFlow. Most of the columns correspond to columns of the same name on the Flow Table sheet. A few columns need additional explanation:

Row	
Label	
Enable	
Group	A read-only field showing the group qualifier, made up of the Group Specifier, Sense, Condition, and Name column values. This is a concatenated value and cannot be edited in MiniFlow.
Device	A read-only field showing the device qualifier, made up of the Device Sense, Condition, and Name column values. This is a concatenated value and cannot be edited in MiniFlow.
Opcode	
Parameter	

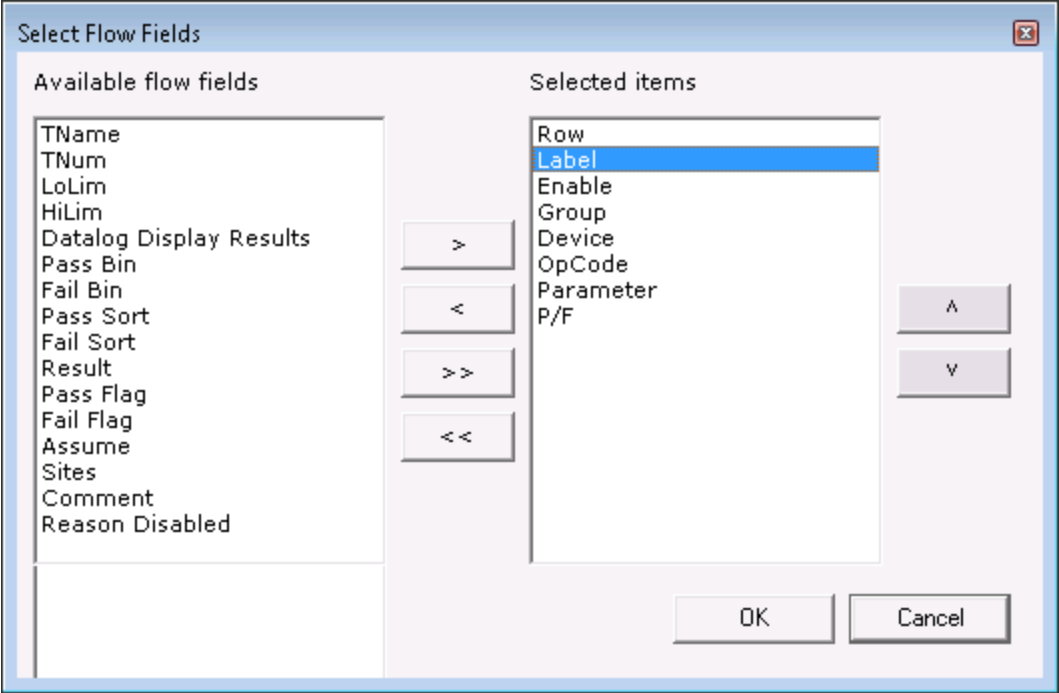
P/F The result of executing the test flow step. This value is set after the flow step has been executed. The pass/fail status is also indicated by the color of the test name after the step has been executed.

You cannot remove the following columns from the display: Label, Enable, Opcode, and Parameter. You can remove the others, and you can also change the order in which all of these rows appear.

Adding, Removing, and Reordering Columns

You can specify what columns are to be displayed and in what order they appear.

 Click this button to bring up the Select Flow Fields window:



Select Flow Fields Window

Select the fields and use the > and < buttons to move them between the panes. Use >> and << to move all the fields in one pane to the other pane. In the Selected items pane, use the ^ and v buttons to reorder the fields you have selected.

Most of these columns correspond to columns of the same name on the Flow Table sheet. The following columns may need additional explanation:

Datalog Display Results	A read-only field showing the values of the Datalog Display Results Scale, Units, and Format columns. This is a concatenated value and cannot be edited in MiniFlow.
Reason Disabled	The name of the column that has caused the flow step to be disabled. This also indicated by a red square in the column that caused the step to be disabled. See Step Not Executed (Reason Disabled). The Reason Disabled column is a text version of the red square.

When you save a toolset, your currently selected columns and order are saved with the toolset.

< >		
	2015 Teradyne, Inc. All rights reserved.	

tdtools.2.07

<

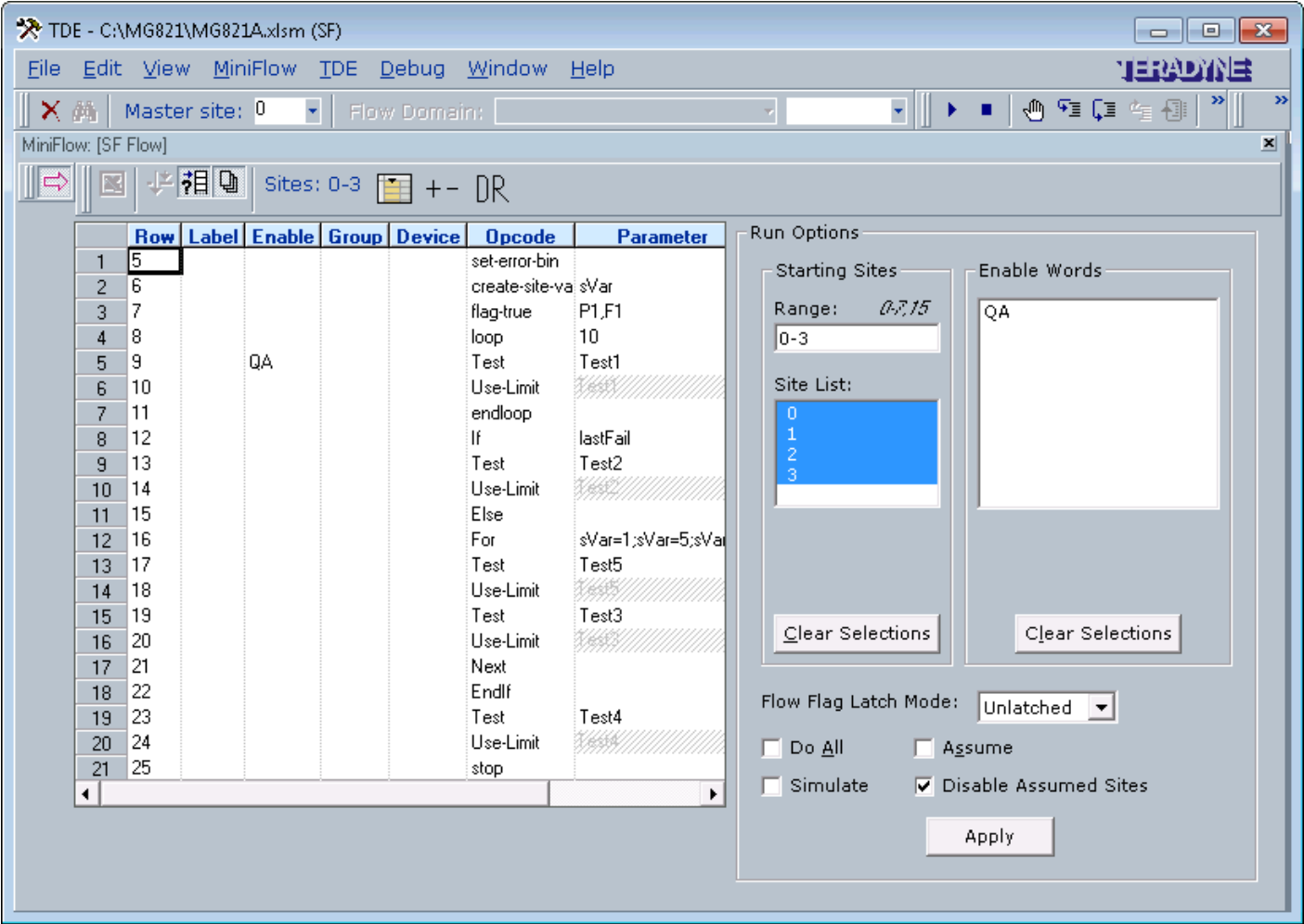
>

TDE Tools : MiniFlow : Run Options Panel

Run Options Panel

For convenience, you can open a Run Options panel as part of the MiniFlow tool window, so that you can set the options that you need to debug the flow. The MiniFlow Run Options panel is a subset of the Run Options window called from DataTool.

 Use this button or the command MiniFlow>Run Options to toggle between showing and hiding the panel. The Run Options panel looks like this:



Run Options Panel in MiniFlow

 Note:

If you change Starting Sites or Enable Words, you must click Apply before changes are applied because these changes affect what is displayed in TDE. You need not click Apply for the drop-down and check box controls under the Starting Sites and Enable Words windows. In some cases, you may need to enlarge the TDE window to make the Apply button appear.

The available options are a subset of those on the DataTool Run Options window, those most relevant to debugging a flow:

- | | |
|---|---|
| • | Select the starting sites. |
| • | Enable or disable individual enable words. See Using Enable Words. |
| • | Enable or disable Do All. |
| • | Enable or disable assume and simulate. For information on using these in MiniFlow, see Assume and Simulate. |

One option not in the DataTool Run Options window is Flow Flag Latch Mode. This option lets you specify whether a flag, once set by a test, can be reset by a subsequent test. The options are:

Unlatched	The flag’s setting can be changed, based on the result of the most recent test step. Default. In Unlatched mode, the flow flag is set according to the test result, even if the flag was set in a previous test step.
TestResult	Flow flags are latched to the test result: the pass flag and the fail flag, once set to True, are not reset by subsequent test steps.
TestFailure	Flow flags are latched to a test failure: the fail flag, once set to True, and the pass flag, once set to False, are not reset by subsequent test steps.

Note that the state of run options is not saved as part of the toolset. You must specify nondefault run options before running a program for the first time.
For more information, see Latched and Unlatched Flag Modes.
The options from the DataTool Run Options window are also available when you run in MiniFlow. See Selecting Run Options in the DataTool section. Note that if you select Detailed Execution Time

on the Setup tab and then run from MiniFlow, the MiniFlow P/F column does not display values.
See [Detailed Execution Time](#).

< >		
	2015 Teradyne, Inc. All rights reserved.	

tdtools.2.08

	
---	---

TDE Tools : [MiniFlow](#) : Row Display

Row Display

The following sections give details on what rows are displayed and how they are displayed.

Gated Rows

[MiniFlow](#) displays only those rows for tests that are potentially executable based on the current settings of the Gate Part, Gate Env, and Gate Job columns in the Flow Table sheet. If the current setting of any of these columns would exclude a row from execution, the row does not appear in [MiniFlow](#). For more information, see [Test Selection by Gating](#) in the DataTool section.

Row Numbers

Row numbers in [MiniFlow](#) correspond to row numbers in the Flow Table. Nonconsecutive row numbers indicate that rows have been omitted because their gating values have excluded them from execution. They can also indicate blocks of flow steps that are currently collapsed: you can cause these blocks to be displays, as described in the next item.

The numbers in the column headed Row start with 5, which is the first data row on the Flow Table sheet. The numbers in the column to the left start with 1, which is the numbering without [MiniFlow](#).

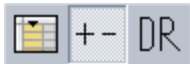
Grouped Steps

By default, [MiniFlow](#) displays all rows without any grouping. There are no controls for the groups of steps: for a Test row and its Use-Limit rows, or the block of flow steps after an If, Elseif, or Else opcode or within a loop:

	Row	Label	Enable	Opcode	Parameter	P/F
1	5			set-error-bin		
2	6			create-site-var	sVar	
3	7			For	sVar = 1;sVar<5;sVar++	
4	8			Test	Test1	
5	9			Use-Limit	Test1	
6	10			If	lastFail	
7	11			Test	Test2	
8	12			Use-Limit	Test2	
9	13			If	lastFail	
10	14			Test	Test3	
11	15			Use-Limit	Test3	
12	16	DoTest4		Test	Test4	
13	17			EndIf		
14	18			EndIf		
15	19			Next		
16	20			Test	Test5	
17	21			Use-Limit	Test5	
18	22			set-device		

Rows without Grouping

To turn grouping on and off, use the + - button on the toolbar:



Grouping Button

When you press the button, group controls appear:

1	2	3	Row	Label	Enable	Opcode	Parameter	P/F
			1	5		set-error-bin		
			2	6		create-site-var	sVar	
			3	7		assign-site-var	sVar 1	
			4	8		For	sVar = 1;sVar<5;sVar++	
			5	9		Test	Test1	
			6	10		Use-Limit	Test1	
			7	11		If	lastFail	
			8	12		Test	Test2	
			9	13		Use-Limit	Test2	
			10	14		If	lastFail	
			11	15		Test	Test4	
			12	16		Use-Limit	Test4	
			13	17		EndIf		
			14	18		EndIf		
			15	19		Next		
			16	20		Test	Test3	
			17	21		Use-Limit	Test3	
			18	22		set-device		

Groups Expanded

You can use the + control to expand a single collapsed group, and then the - control to collapse it again. The numbers to the left of the column headers let you specify the level to which you want groups expanded.

To expand or collapse all groups, use the command View>Groups, which has a pull-right menu containing the options Expand Groups and Collapse Groups. The + - button must be selected for these commands to work.

<		>	
	2015 Teradyne, Inc. All rights reserved.		

tdtools.2.09

<	>
--------------------------------------	--------------------------------------

TDE Tools : [MiniFlow](#) : Debugging Using MiniFlow

Debugging Using MiniFlow

MiniFlow is central to TDE debugging because it allows you to step through the flow. The following topics describe how to debug with MiniFlow:

- [Setting Breakpoints and Stepping Through the Flow](#)
- [Displaying Results and Sites](#)
- [Adding and Deleting Rows and Editing Values](#)
- [Using Enable Words](#)
- [Step Not Executed \(Reason Disabled\)](#)
- [Bin Tables and Subflows](#)
- [Assume and Simulate](#)
- [Using the Find and Filter Commands](#)
- [Displaying Test Name, Number, and Limits](#)
- [Using the Find and Filter Commands](#)

< >		
	2015 Teradyne, Inc. All rights reserved.	

tdtools.2.10



TDE Tools : MiniFlow : Setting Breakpoints and Stepping Through the Flow

Setting Breakpoints and Stepping Through the Flow

You can set or clear a breakpoint by clicking in the MiniFlow margin to the left of the row numbers.



You can also use the TDE Toggle Breakpoint button or the command on the Debug menu.

You can step through a test program using TDE debug functions.

To start and stop execution, use these buttons:

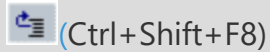
Debug Run	 (F5) Execute the flow to the next breakpoint or to the end of the flow.
Debug Stop	 Stops execution. You can use this when you are stopped on a flow step. The rest of the flow is not executed.

You can also use Step Over or Step Into to start execution and pause at the first flow step.

When execution is paused before a test, the TDE toolbar and TDE Debug menu have these options for executing the test:

Step Over	 (F8) Executes the entire test as a unit and then pauses before the next step in the flow.
Step Into	 (Shift+F8) Steps into the current test so that you can execute its parts separately. The action depends on what the test is based on. See the description below.

Step Out



(Ctrl+Shift+F8)

If you use Step Out when paused before a test, execution runs to the next breakpoint or to the end of the flow. Step Out is most useful with steppable tests, as described below.

For more information, see [Debug Menu](#).

Stepping Into a Test

The action of the Step Into function depends on the kind of test or flow step on which execution is paused.

If the flow step is an If/Else/Elseif block, Step Into expands the collapsed block and steps through the flow steps in the block.

If the test is based on a VBT test, Step Into brings up the VBT Debugger (if it is enabled) or the VBA Editor with the code open and execution paused at the first statement in the test.

A test flow step that uses Use-Limit steps is by default collapsed. Note that using Step Into when on the test flow does not expand the Use-Limit rows, because the Use-Limit rows are never actually executed. Step Into instead steps into the VBA editor.

If the test is based on a test procedure, Step Into brings up PDE with the test procedure open and execution paused before the first test element.

If a test is based on a steppable test, such as the tests in the Teradyne-supplied VBT test library, Step Into first displays three horizontal bars to indicate the Prebody, Body, and Postbody of the template, with execution paused before the bar representing the Prebody:



Debugging a Steppable Test

When execution is paused before any part of a three-part steppable test, and the three-bar symbol appears, you have these options:

Step Into



Brings up the Visual Basic editor, displaying the VBT code for the Prebody, Body, or Postbody. In VB, you can execute each step of the code or use Step Out to return to the next part of the test in MiniFlow.

Step Over



Executes the next part of the test without stepping into the template code.

Step Out



Executes the remaining parts of the test, steps out of the test, and runs to the next breakpoint or to the end of the flow.

➡ While you are stepping through tests, a yellow arrow indicates the point at which execution is stopped. The arrow indicates the next flow step to be executed.

<		>	
	2015 Teradyne, Inc. All rights reserved.		

tdtools.2.11

<

>

TDE Tools : MiniFlow : Displaying Results and Sites

Displaying Results and Sites

By default, as you step through a flow step, MiniFlow shows the results of executing the step on the site specified as the Master Site in the TDE toolbar: If the site passed the test, MiniFlow colors the test name green; if it failed, it colors the name red.

If addition, if you have included the P/F column in the display, MiniFlow displays a P or F for each test step. This is a text version of the information conveyed by the color of the test name.

The following shows the results for site 0, the site specified by the TDE Master Site control:

Master site: 0

Flow Domain:

MiniFlow: [FlowMS1950\Test5]

Sites:

		Row	OpCode	Parameter	P/F
+	4	8	Test	Test1	P
+	6	10	Test	Test2	P
+	8	12	Test	Test3	F
+	10	14	Test	Test4	P
+	12	16	If	sv1 < 2	
	15	19	Else		
	18	22	EndIf		

Results for Site 0

If you use the Master site control to change the active site, MiniFlow displays the results for that site, in this case, site 3:

Master site: 3

Flow Domain:

MiniFlow: [FlowMS1950\Test5]

Sites:

		Row	OpCode	Parameter	P/F
+	4	8	Test	Test1	P
+	6	10	Test	Test2	F
+	8	12	Test	Test3	P
+	10	14	Test	Test4	P
-	12	16	If	sv1 < 2	
	13	17	Test	Test5	
	14	18	Use-Limit		

Results for Site 3

If the current master site is binned out, MiniFlow switches the Master site and the display to the lowest possible remaining site.

The Show All Sites Result button on the MiniFlow toolbar lets you toggle between showing results only for the site specified by Master site and showing results for all sites. When the results for all sites are displayed, the flow step is colored red if any sites failed the flow step. It is colored green if all sites passed the flow step.

When you save a toolset, the state of the Show All Sites Result option is saved with the toolset.

<

>

2015 Teradyne, Inc. All rights reserved.

tdtools.2.12

<	>
-----------------	-----------------

TDE Tools : MiniFlow : Adding and Deleting Rows and Editing Values

Adding and Deleting Rows and Editing Values

MiniFlow lets you make most of the modifications that you can make to a Flow Table sheet in DataTool.

You can insert a row before any flow step. Select the flow step, right-click, and from the context menu select Insert Test. You can also use Edit>Insert Test. A new row is inserted with the Opcode set to nop. All opcodes are available on the drop-down list.

To delete a row, select the row and use Delete Test from the context menu or Edit>Delete Test.

Most MiniFlow cells are editable. Columns that have a set of predefined values, such as Opcode and Result, have drop-down lists. For others, you click in the cell and enter a value.

Some MiniFlow columns are read-only. These include columns that are concatenated from multiple Flow Table sheet columns, such as Device and Group, as well as MiniFlow-only columns such as Reason Disabled. You cannot edit these columns in MiniFlow.

Any change you make to in MiniFlow is immediately written to the Flow Table sheet. Revalidation is required before you can execute the flow.

<	>	
	2015 Teradyne, Inc. All rights reserved.	

tdtools.2.13

<	>
-----------------	-----------------

TDE Tools : [MiniFlow](#) : Using Enable Words

Using Enable Words

The Enable column always appears in the MiniFlow.

You can enable and disable enable words using the [Run Options](#) pane. See [Run Options Panel](#).

Note that you must click Apply for any changes to apply.

You can also right-click the word in the Enable column and select Set Enable Word or Clear Enable Word. If the Enable column contains a logical expression other than ! (not), these options are disabled. These options are also available from the TDE Edit menu.

<	>	
	2015 Teradyne, Inc. All rights reserved.	

tdtools.2.14



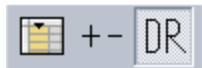
TDE Tools : MiniFlow : Step Not Executed (Reason Disabled)

Step Not Executed (Reason Disabled)

A flow step may not be executed for the currently selected site for several reasons: the step's enable word may not be enabled, or it may be in an If block that does not evaluate to True for this site.

When you are stepping through the flow, it can be useful to know why step has not been executed. To enable this MiniFlow feature, perform these actions:

- On the toolbar, click the DR button:



DR Button

This enables the display of the Disabled Reason.

-  Add the Reason Disabled column to the display. Click this button to bring up the Select Flow Fields window, and move Reason Disabled from the Available list to the Selected list. See [Adding, Removing, and Reordering Columns](#).

If you press the DR button, MiniFlow grays out the steps that are not executed. For steps with an Enable word, if the word is not enabled, the step is grayed out at validation. Other steps may not be executed because a device qualifier, group qualifier, or If condition evaluated to False for the site. In this case, the step turns gray only after you have stepped over the flow step and the qualifier or condition has been evaluated. MiniFlow also displays a red square in the column that caused the flow step not to be executed:



Red Square in MiniFlow

If you include the Reason Disabled column, MiniFlow also displays the reason in the column. The reason can be:

Enable	The enable word is not enabled.
Group or Device	The group or device qualifier on the flow step does not evaluate to True for the site.
Opcode	The If or Elself condition does not evaluate to True for this site.

(If you include the Reason Disabled column without pressing the DR button, the column remains blank. You must press the button for the reason to be displayed.)

For example, in the following flow, the enable word enableW is not enabled, and the device qualifier and If condition both test whether a site variable is set to 2, when it is set to 1:

	Row	Enable	Group	Device	OpCode	Parameter	P/F	Reason Disa	Label
1	5				set-error-bin				
2	6				create-site-va	sv1			
3	7				assign-site-va	sv1 1			
4	8				Test	Test1	P		
6	10	enableW			Test	Test2		Enable	
8	12			site-var= sv1 2	Test	Test3		Device	
10	14				Test	Test4	P		
12	16				If	sv1 = 2		OpCode	
13	17				Test	Test5			
14	18				Use-Limit	Test5			
15	19				Else				
16	20				Test	Test6	P		
18	22				EndIf				
19	23				Test	Test7			
21	25				set-device				

Reason Disabled

The red squares indicate which column in the flow step disabled the test. The Reason Disabled gives the same information as text. Note that, for the If condition, the red square and reason are on the same step as the If, while it is the subsequent Test flow step that is actually disabled.

Note that you can enable or disable an Enable word from the right-click context menu, even while stepping through the flow. Select the flow step containing the Enable word and right click. If the word is currently disabled, the menu contains the option Set Enable Word. If the word is currently enabled, the option is Clear Enable Word. The selected option takes effect immediately.

<

>

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

tdtools.2.15

<

>

TDE Tools : [MiniFlow](#) : Bin Tables and Subflows

Bin Tables and Subflows

If execution is paused on a Bintable or call opcode, you can step over or into the bin table or subflow:

[Step Over](#) executes the specified bin table or subflow in a single step, without stepping into the bin table or subflow. You can see the results of the binning or subflow in the Watch tool. See [Watch](#).

[Step Into](#) on the Bintable opcode brings up the MiniBin tool and steps through the bin table. See [MiniBin](#).

The effect of Step Into on the call opcode depends on whether the subflow is on a different Flow Table sheet or on the same Flow Table sheet.

The following shows a call to a subflow that is on a different Flow Table sheet:

1

→

+

+

+

+

Row	Label	OpCode	Parameter
1	5	Test	Leakage
2	6	Test	Continuity
3	7	call	FourTest
4	8	call	TwoTest
5	9	Bintable	Binning
6	10	set-device-ne	
7	11	stop	
8	12	TwoTest	Test5
10	14	Test	Test6
12	16	return	

MiniFlow Before Call to Subflow

[Step Into](#) brings up another MiniFlow window for the called subflow:

1

→

+

+

+

+

Row	OpCode	Parameter
1	5	Test4
3	7	Test3
5	9	Test2
7	11	Test1
9	13	return

Subflow on a Different Sheet

If you use Window>MiniFlow to look at the original MiniFlow window, a green arrow indicates the location where the subflow was called and that the current execution point is on a different sheet:

1

	Row	Label	OpCode	Parameter
1	5		Test	Leakage
2	6		Test	Continuity
3	7		call	FourTest
4	8		call	TwoTest
5	9		Bintable	Binning
6	10		set-device-ne	
7	11		stop	

MiniFlow with Green Arrow

A green arrow also appears in the MiniFlow when execution is currently stopped in MiniBin on a binning step, or in PDE on a test procedure step. When the green arrow appears, you cannot control execution from the MiniFlow. You must go to the other TDE tool to step through the procedure there.

The following shows a call to a subflow that is on the same Flow Table sheet:

1

	Row	Label	OpCode	Parameter
1	5		Test	Leakage
2	6		Test	Continuity
3	7		call	FourTest
4	8		call	TwoTest
5	9		Bintable	Binning
6	10		set-device-ne	
7	11		stop	
8	12	TwoTest	Test	Test5
10	14		Test	Test6
12	16		return	

Call to Subflow on Same Sheet

Step Into moves the execution point to the first step of the subflow:

1

	Row	Label	OpCode	Parameter
1	5		Test	Leakage
2	6		Test	Continuity
3	7		call	FourTest
4	8		call	TwoTest
5	9		Bintable	Binning
6	10		set-device-ne	
7	11		stop	
8	12	TwoTest	Test	Test5
10	14		Test	Test6
12	16		return	

Subflow on Same Sheet

If you have called subflows, you can use the FlowCallStack tool to see the call sequence and display any instance of MiniFlow in the sequence. See FlowCallStack.

<

>

tdtools.2.16

<

>

TDE Tools : MiniFlow : Assume and Simulate

Assume and Simulate

MiniFlow provides a way to see how complex flows will execute under different conditions. When debugging test program flow, result data is not required. What is important is to determine how a pass or fail result in one flow step influences the subsequent steps, for example, in terms of evaluating If conditions and device and group qualifiers.

The Assume feature lets you assume pass/fail results per site. You specify, for a test, which sites should pass or fail. With this capability, you need not execute any VBT code because the results are assumed and are not based on limits checks. The Simulate feature lets you execute the flow without executing any VBT code. When run in this mode, all tests are assumed to pass unless you explicitly assume fail.

To make the results easier to see, MiniFlow uses different colors for a pass/fail result that comes from actually executing VBT code and a pass/fail result that is based on an assume specification:

	Row	Label	OpCode	Parameter	Resu	Assume	Sites
1	5		set-error-bin				
2	6		Test	Test11		Fail	1
5	9		Test	Test12		Pass	1
8	12		Test	Test13			
11	15		Test	Test14			
14	18		set-device				

Colors for Assumed Pass and Fail

The colors are as follows:

Dark green	Actual pass.
Light green	Assumed pass.
Red	Actual fail.
Pink	Assumed fail.

For more information, see [Assume and Simulate Features](#) in the DataTool section on flow programming.

<		>	
	2015 Teradyne, Inc. All rights reserved.		

tdtools.2.17

<	>
-----------------	-----------------

TDE Tools : MiniFlow : Displaying Test Name, Number, and Limits

Displaying Test Name, Number, and Limits

You can define the test limits for a test instance in a Use-Limit row on the Flow Table sheet. In addition, you can define the test name and test number either on the Test row or on the Use-Limit row.

You can add the relevant columns to the MiniFlow display. In addition, you can display these values by placing the pointer over the test instance name in the Parameter column of the row where they are defined, either a Test row or a Use-Limit row. The tooltip contains the test instance name plus the other values defined in the row.

If the values are defined on the Test row, they are not displayed if you point at the Use-Limit row. If they are defined in the Use-Limit row, they are not displayed if you point at the Test row. Note that you must point at the test instance name in the Parameter column. Pointing at other columns does not display the values.

<	>	
	2015 Teradyne, Inc. All rights reserved.	

tdtools.2.18

<

>

TDE Tools : MiniFlow : Using the Find and Filter Commands

Using the Find and Filter Commands

When debugging a test, you may want to look at similar tests. To find similar tests in the flow:

1. Select a test instance row in MiniFlow. The row must contain the opcode Test.
2. On the Edit menu, select Find to bring up the Find Tests dialog box.

Note that Find is enabled only if you are not currently stepping through the flow.

3. Select check boxes in the Find Test box to specify the criteria that tests must match to be displayed.

For example, you can specify tests that match the current test’s AC Context or Pin Levels sheet.

4. Click OK in the dialog box, and only matching tests are displayed.

The dialog box includes these controls:

Test Procedure Type	Matches any test instance that is of the same type as the currently selected test instance. See Instance Type Column for the Test Instances sheet in the DataTool section.
Argument contains	Matches any test instance that has an argument containing the string entered here. The string can match part or all of any of the test instance’s arguments. Note that this string may not match the currently

selected test instance, if it does not have this string among its arguments.

 Once you apply a Find, the Filter function is enabled, as a button and as a command on the View menu. Filter acts as a toggle to display either all tests or only tests that match the Find criteria. If you select a different “Test” row and toggle the filter, the Find criteria are applied to the new test, and only tests matching the new test are displayed.

			
	2015 Teradyne, Inc. All rights reserved.		

tdtools.2.19



TDE Tools : MiniFlow : Commands and Toolbar Buttons

Commands and Toolbar Buttons

In addition to the standard TDE commands, MiniFlow provides a set of commands and toolbar buttons. Commands are on the following menus:

- File Menu
- Edit Menu
- View Menu
- MiniFlow Menu
- Debug Menu

The following information and functions are also available:

- Starting Sites Display
- Right-Click Menu

For more information, see [Standard TDE Commands and Buttons](#).

File Menu

In addition to the standard TDE File commands, MiniFlow provides these commands:

Print	Prints the MiniFlow display.
Export Flow	Writes the MiniFlow display to a tab-delimited ASCII file. A browser lets you specify the output file.

Edit Menu

Clear	 Clears the currently selected cell.
Find	 (Ctrl+F) Finds tests that match specified characteristics of the current test. The function is enabled only if you have selected a row that has the "Test" opcode. See Using the Find and Filter Commands.
Set Enable Word, Clear Enable Word	Sets or clears the enable word specified in the currently selected row. If an enable word is set, the row is executed. See Enable Column in the DataTool section. Set Enable Word is enabled only if the row contains an enable word that is not set (displayed in gray). Clear Enable Word is enabled only if the row contains an enable word that is set (displayed in black).
Test Instance, Test Procedure, Characterization Setup, VBT	Brings up the tool for editing the test instance (instance editor), test procedure (PDE), characterization setup (Characterization Studio), or VBT code (VBE) for the test in this row. Only options for the item in the row are enabled.
Insert Test	Inserts a row into the flow, above the currently selected row. The opcode is set to nop.
Delete Test	Deletes the currently selected row from the flow.

View Menu

Filter	 Toggles the filter on and off. Use Find to set up the filter. See Using the Find and Filter Commands.
--------	--

Font Size	Changes the font size of the text in the grid (but not of the column headers). Options are Small, Medium, and Large. You can include more information in the MiniFlow display by reducing the font size.
Groups	Expands or collapses the grouped rows in the flow. The pull-right menu has the options Expand All and Collapse All.

MiniFlow Menu

Run Options	 Brings up the Run Options panel to specify options for running the program. See Run Options Panel.
Go to Excel	 Goes to the row of the Flow Table sheet that defines the currently selected row.

Debug Menu

Debug functions are as follows. See also [Debugging Using MiniFlow](#) in the Mini Tools section.

Run	 (F5) Begins running the program or continues execution from the current stopping point to the next breakpoint or the end of the flow. See Selecting Run Options.
Stop	 Stops program execution in debug mode. See Debug Run and Debug Stop.
Step Into	 (F8) Executes the current item one line at a time by stepping into the lower-level code: for example, by stepping into a test template or test procedure from the flow. If there is no lower-level code, Step Into is equivalent to Step Over.
Step Over	 (Shift+F8) Executes the next item in the flow without stepping into the current item. See Step, Step Over, and Step Out.

Step Out



Executes the remaining lines of the current item and then steps back up to the next higher level: for example, from a test procedure back to the flow. See [Step](#), [Step Over](#), and [Step Out](#).

[Step Out](#) is enabled only if there is a higher level to which execution can return.

Loop



Executes the remaining lines of the current item and then steps back up to the next higher level: for example, from a test procedure back to the flow. See [Step](#), [Step Over](#), and [Step Out](#).

[Step Out](#) is enabled only if there is a higher level to which execution can return.

Toggle Breakpoint



Sets or removes a breakpoint on the current row. You can also set or clear a breakpoint by clicking in the MiniFlow margin to the left of the row number. See [Toggle Breakpoint](#).

Clear MiniFlow Breakpoints

Removes breakpoints from MiniFlow and from the corresponding Flow Table sheet, but does not remove breakpoints in procedure code. See [Toggle Breakpoint](#).

Clear All Breakpoints

(Ctrl+Shift+F9)

Removes all breakpoints in the flow and in all procedure code. See [Toggle Breakpoint](#).

Call Stack, Set Next, Run to Element

These commands are part of a legacy IG-XL feature that is not recommended for new test development. See [Note on Test Procedures, Test Elements, and PDE](#) in the DataTool section.

Starting Sites Display

The MiniFlow toolbar includes the following read-only display:

Sites: 0, 1

(Starting Sites)

Displays the starting sites, as specified by [Run Options](#). By default, all sites in the channel map are in the starting site list.



To change the starting sites, click the [Run Options](#) button to bring up the Run Options panel. See [Run Options Panel](#).

Right-Click Menu

Some MiniFlow commands are available from the right-click menu. The commands apply to the currently selected row. The commands are:

Edit	Lets you edit the test instance, test procedure, characterization setup, or VBT code for the test in the current row. The right-click menu has valid editing options enabled. See Edit Menu.
Definition	Goes to the selected DataTool sheet. When you right-click the test opcode row, the Definition option lists the DataTool sheets that define values for the test, or to the VBE module where the test is defined. This is like the Definition option in DataTool: see Quick Navigation from Right-Click Menu in the DataTool section.
Set Enable Word, Clear Enable Word	Sets or clears the enable word in this row, which enables or disables execution of the row. See Edit Menu.
Insert Test, Delete Test	Inserts a new row into the flow before the currently selected row or deletes the currently selected row from the flow. See Edit Menu.

<	>	
	2015 Teradyne, Inc. All rights reserved.	

tdtools.2.20



TDE Tools : PSet and Signals Editor

PSet and Signals Editor



This is the icon for the PSet and Signals Editor.

The PSet and Signals Editor (PSSE) lets you view, edit, and create definitions of PSets and signals, both during run time and offline. You can define PSets and signals on the PSets or Signals sheets in DataTool, or create them in VBT. The PSSE can display the definitions that were created on the sheets and those that were created dynamically in VBT, and can also indicate any differences when one item has both a sheet definition and a dynamic definition in the driver.

In addition, the PSSE lets you generate VBT code that creates a PSet or signal based on the definition currently displayed.

You can invoke the PSSE from the DataTool or TDE toolbar. You can also invoke it from an instrument debug display or from PatternTool to view or modify a specific PSet or signal in the context in which it is used.

The following is the PSSE showing two PSets defined for a VHFAC Source pin:

Source:

Pin names:

Instrument:

Add/Remove PSets...

☐ Sheet definition ☒ Current driver definition

PSet parameters:

Name	pset250mv	pset500mv
Amplitude Unit	VoltPeak	VoltPeak
Amplitude	0.25	0.5
Calibration Frequency	15000000	15000000
Common Mode for AC	0	0
Common Mode for DC	0	0
Low Pass Filter Bypass	FALSE	FALSE
Low Pass Filter Bandwidth	20M	20M

Update Tester Update Sheet Apply PSet

General Errors

PSet and Signals Editor

The following topics are included:

- [Sheet and Driver Definitions](#)
- [Invoking the PSSE](#)
- [Specifying PSets and Signals for Display](#)
- [Viewing PSets and Signals](#)

- [Editing and Updating PSets and Signals](#)
- [Creating, Copying, and Saving PSets and Signals](#)
- [Displaying PSets and Signals Side-By-Side](#)
- [Generating VBT Code](#)
- [PSSE Buttons and Commands](#)

<

>

2015 Teradyne, Inc. All rights reserved.

tdtools.2.21

<

>

TDE Tools : PSet and Signals Editor : Sheet and Driver Definitions

Sheet and Driver Definitions

The PSSE interface makes a basic distinction between sheet definitions and current driver definitions. For example:

Source: PSet

Pin names: Ain_P

Instrument: VHFACSource

Add/Remove PSets...

Sheet definition

Current driver definition

Sheet and Driver Definitions

You can define PSets and signals in two ways:

- On a PSets sheet and Signals sheet in DataTool
- Dynamically using VBT

Using VBT, when a VBT statement is executed, the definition is loaded into PSet or signal memory: this is known as the "driver definition." In addition, when the workbook is validated, the sheet definition is also loaded into memory, so that the sheet definition becomes part of the driver definition as well.

The PSSE can display either the sheet definition or the current driver definition, depending on the setting of the radio button:

Sheet definition	Includes only definitions found on the PSets or Signals sheet.
------------------	--

Current driver definition	Includes both sheet definitions that have been loaded into memory on workbook validation and VBT definitions that have been loaded into memory on execution of the VBT statements.
---------------------------	--



tdtools.2.22

<

>

TDE Tools : PSet and Signals Editor : Invoking the PSSE

Invoking the PSSE

To invoke the PSSE, select Tools>PSet and Signals Editor in the Launch group on the IG-XL Tools tab. This brings up TDE with the editor with no PSets or signals displayed. You can also invoke the PSSE from a debug display and from PatternTool, to view a PSet or signal in the context in which it is used:

- If an instrument supports signals and PSets, its debug display has controls that list the available signals and PSets and that display the currently selected one. The Edit button under the signal and PSet controls let you bring up the PSSE so that you can view the signal or PSet that the control specifies.

Signal: PRMLWave_Signal

View

Edit

Load Settings

Start Immediately

Stop Immediately



Calling the PSSE from a Debug Display

Note that several instruments support PSets but do not allow you to access PSets in the PSSE from the debug display. Those instruments include HDVS, HexVS, UltraVS256, and VSM.

- In PatternTool, if the Command cell has the PSet opcode or the Trigger or Start opcode (for a signal), its right-click menu contains the option Edit PSet/Signal, which brings up the PSSE so you can view the PSet or signal that the opcode specifies.

If you invoke the PSSE when no sites are active (for example, when you have just opened the workbook) the PSets and signals available for display are those defined on the PSets and Signals sheets. Sheet definition is enabled and selected; Current driver definition is not.

For more information, see [Specifying PSets and Signals for Display](#).

<		>	
	2015 Teradyne, Inc. All rights reserved.		

tdtools.2.23

<

>

TDE Tools : PSet and Signals Editor : Specifying PSets and Signals for Display

Specifying PSets and Signals for Display

This section describes how to display one or more PSets or signals that have been defined for the same pin, pin group, or pin list.

The following topics are available:

- [Specifying the Pins and Instrument](#)
- [Selecting PSets or Signals](#)
- [Matching Pins, Pin Groups, and Pin Lists](#)

To display dissimilar items, such as PSets and signals for the same pin or PSets or signals for different pins or for the same pin but different instruments, see [Displaying PSets and Signals Side-By-Side](#).

You can display up to 10 PSets or signals at a time.

The controls for specifying a PSet or signal are at the top of the editor:

Source:

PSet

Pin names:

Ain_P

...

Instrument:

VHFACSource

Add/Remove PSets...

+

-

☐ Sheet definition

☒ Current driver definition

Controls for Specifying an Item

If PSets or signals are already displayed, the controls identify the pin and instrument for which the items are defined.

If you invoke the PSSE from TDE, the tool comes up blank, and you must specify the PSets or signals to be displayed. This section assumes that the PSSE is empty and describes how to specify items to be displayed. It also applies to the case where you are displaying one PSet or signal and want to display additional items for the same pin.

In general, the steps for specifying the items to display are:

1. [Use the Source: control](#) to select PSet or Signal.
2. [Specify the pin and instrument whose PSets or signals you want to display, using the Pin names: and Instrument: fields](#) or the Pin / Group Selector. See [Specifying the Pins and Instrument](#).
3. [The next button is labeled Add/Remove PSet or Add/Remove Signal](#), depending on the value of Source:. Click this button to bring up the PSets / Signal Selector to specify the PSets or signals to be displayed. See [Selecting PSets or Signals](#).
4. [The radio buttons Sheet definition and Current driver definition](#) let you determine which definitions are displayed: those on the PSets or Signals sheet or those that are available to the instrument driver. See [Sheet and Driver Definitions](#).

Specifying the Pins and Instrument

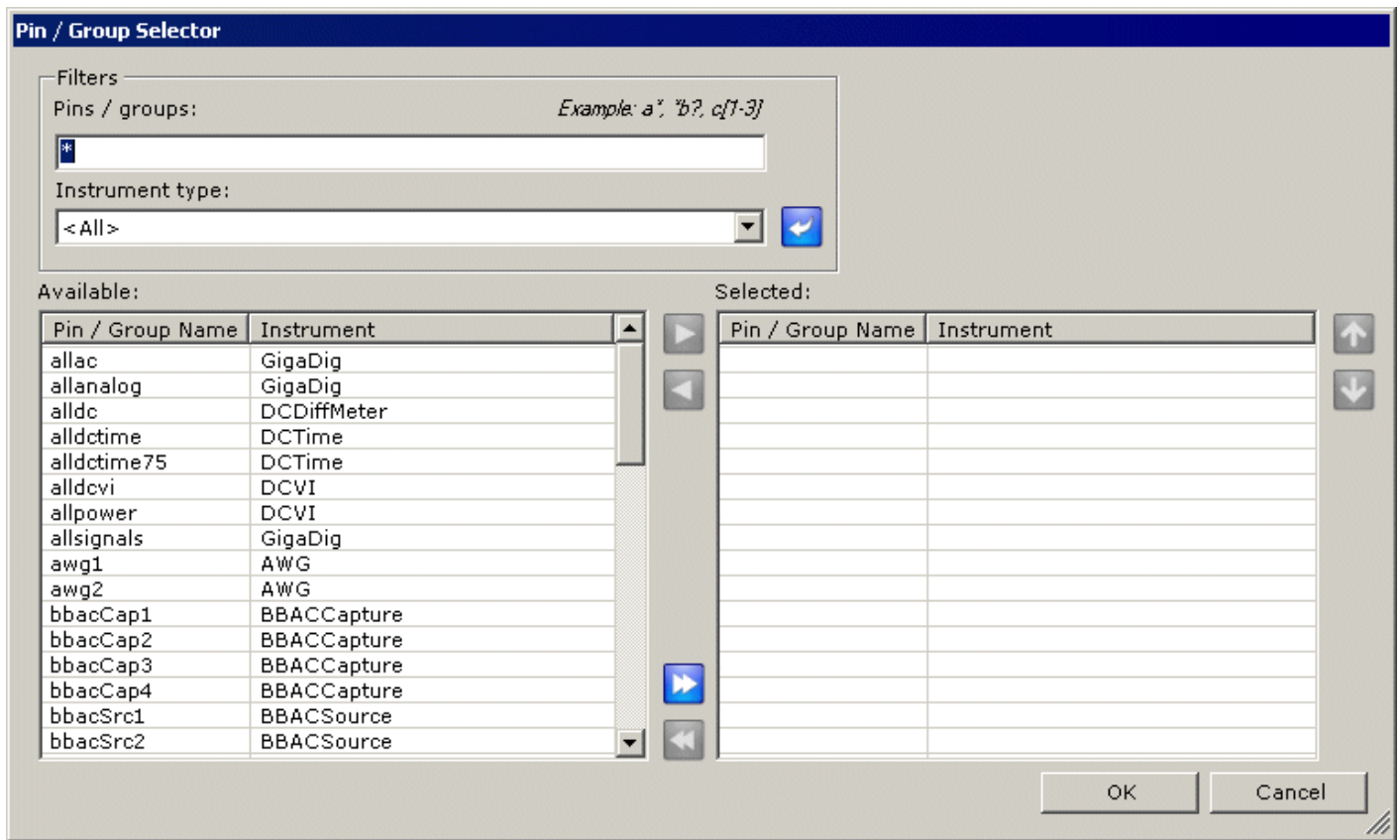
[Before you can display PSets or signals, the Pin names: and Instrument: controls](#) must specify the pin and instrument for which they are defined.

[Use the controls on the display to enter the values or use the Pin / Group Selector window.](#)

[Pin names:](#) can be a single pin name, a pin group name, or a comma-separated list of pin names and pin group names. For more information, see [Matching Pins, Pin Groups, and Pin Lists](#).

[You can directly enter a pin, pin group, or pin list and select an instrument from the drop-down list, which contains instruments that support PSets and signals. If you first type in a pin name, the list contains only instruments connected to the pin.](#)

[You can also use the ellipsis button to the right of Pin names: to bring up the Pin / Group Selector, which allows you to select from all available pins and to use filters to narrow the list. Initially, the window looks like this:](#)



Pin/Group Selector

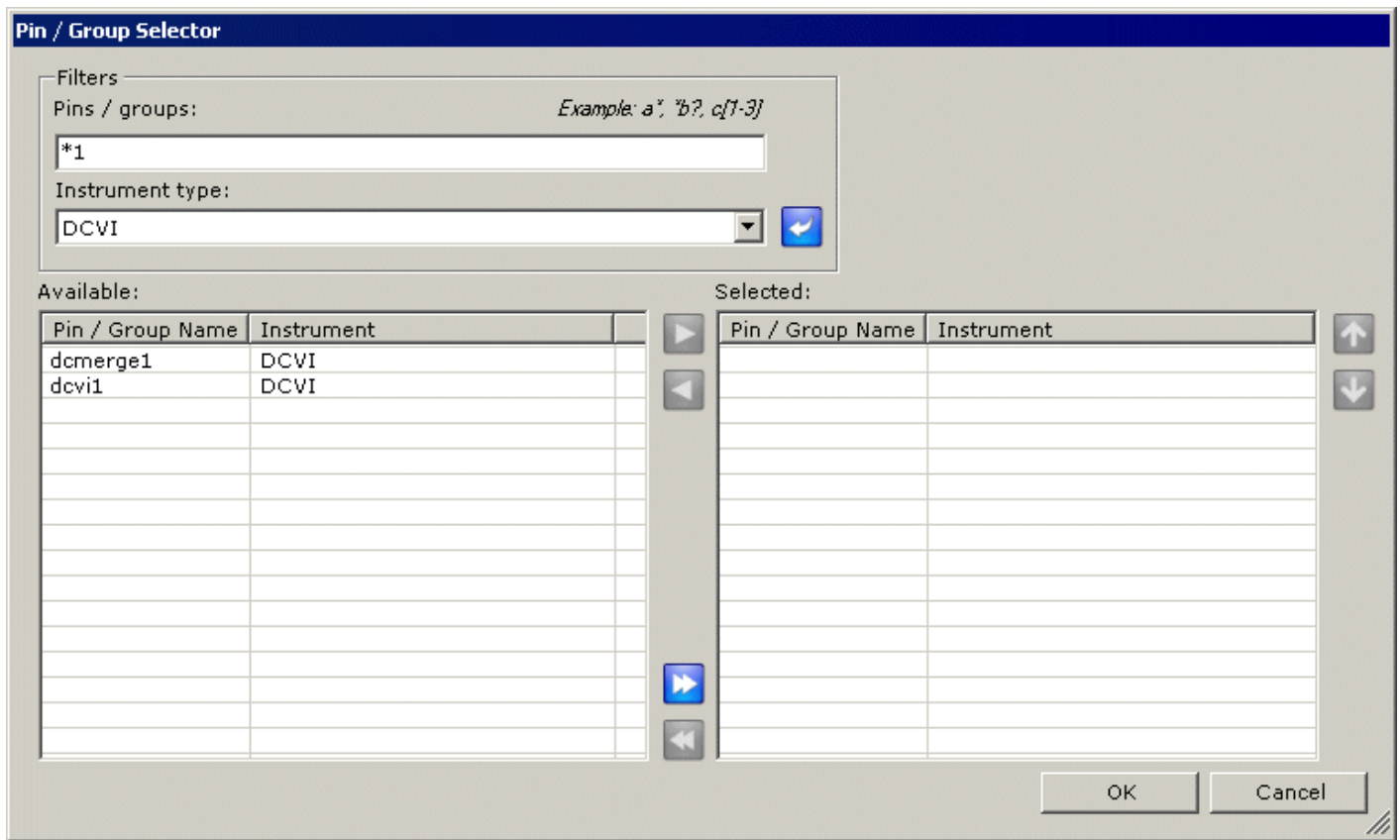
Without any filters, the Selector displays all pins and pin groups connected to instruments that support PSets and signals. To narrow the list, use the filters at the top of the window:

- Pins / groups:** Enter a template using the standard wildcards: *, ?, [n-m], or [n,m]. For example, "bbac*[1,3]" would match any pin whose name starts with "BBAC" and ends with "1" or "3." The default is *, which matches all pin names.
- Instrument type:** Select any instrument type from the drop-down list, to display only pins connected to that instrument. The default is <All>, which displays all the instrument types.

If you select an instrument from the Instrument: drop-down list on the main window before bring up the Selector, that instrument will be already applied as a filter when the Selector comes up.



Once you specify filters, use the blue arrow to apply the filters and display the resulting list of pins and groups:



Pin / Group Selector with Filters

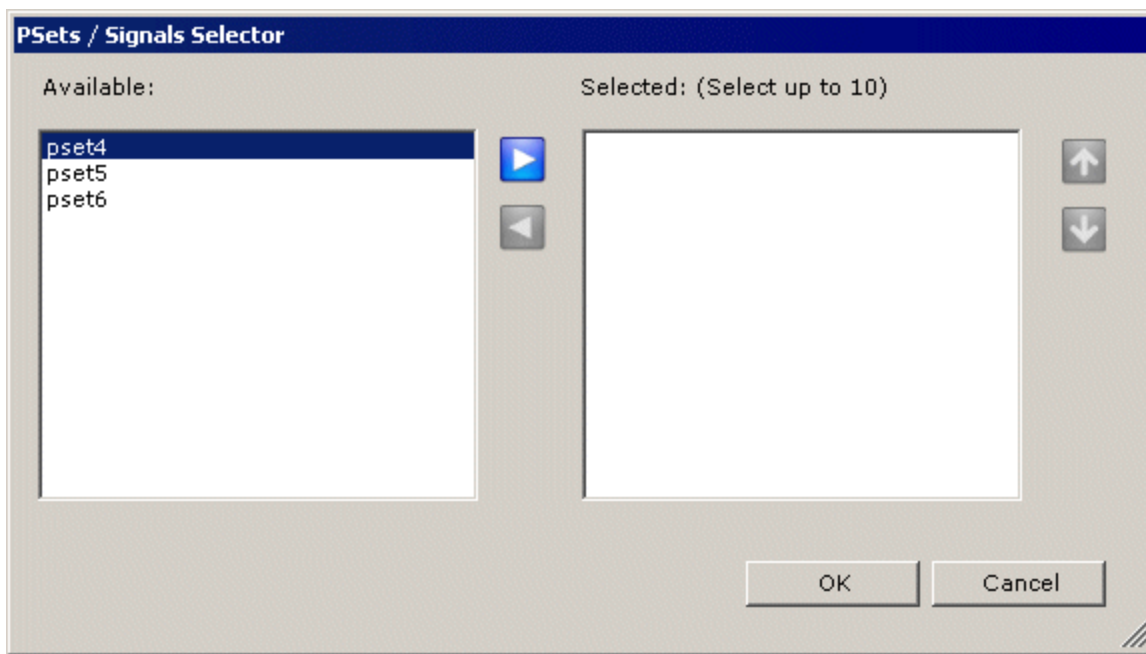
From pins and groups displayed in the Available: window, select the ones you want and use the arrows to move them to the Selected: window. Single arrows move the selected items. Double arrows move all items from one window to the other. You can also double-click an item to move it between windows.

If you select multiple items, they become a comma-separated pin list. See [Matching Pins, Pin Groups, and Pin Lists](#).

Once the pins you want are in the Selected: window, click OK. At this point you return to the main PSSE window, where you can now select from the PSets or signals defined for the selected pins and instrument. See [Selecting PSets or Signals](#).

Selecting PSets or Signals

Once you specify a pin or pins, click the Add/Remove button to bring up the PSets / Signals Selector window. You must have values for both the Pin Names: and Instrument: fields before you can bring up the window. The window looks like this:



PSets / Signals Selector

PSets or signals that are defined for the specified pins are displayed in the Available: window. Select items you want, then click the right-arrow to move them to the Selected: window; you can also double-click an item to move it between windows. You can select up to 10 items. To change the order in which items are displayed, select an item and use the up and down arrows to move it. The top-down order here corresponds to the left-to-right order in the display. Click OK to display the items.

To remove an item from the display, right-click a name in the heading in the display and select Delete from the menu. You can also click Add/Remove, and then select the items and move them from the Selected: back to the Available: window. Note that removing an item removes it from the display; but it does not delete the item from the sheet or driver definitions.

A PSet or signal definition can use multiple rows on the sheet, each row for one pin or pin group. Similarly, PSets and signals defined in VBT can be defined over several pins or pin groups. If you select such a PSet or signal, the PSSE shows only the one row of the definition for the specified pin or pin group.

In some cases you may be unable to open a PSet or signal. As an example, assume that PSet psA is defined for pin group pg1, which includes the pin p0. When you are stopped at a breakpoint, you go into VBT and modify a parameter value for p0. This means that the current driver definition has one value for p0 and a different value for the rest of the pin group. Now, in the PSSE, you specify the pin group name pg1 and try to open the PSet. You get an error message saying that the PSet does not have the same value across all pins and you cannot open the PSet. If you specify a pin in the group instead of the group itself, you can open the PSet with that pin's values.

Matching Pins, Pin Groups, and Pin Lists

A Pin names: value can be a pin, a pin group, or a comma-separated list of pins and group. It can also be a pin that is a member of a pin group. This section describes the effects of these selections.

What is matched depends on whether the PSSE is set to Sheet definition or Current driver definition. The driver definition decomposes pin groups into their constituent pins, while the sheet definition does not. This means that matching is more flexible with the driver definition.

For example, suppose PSet psA is defined for pin group pg1, which includes pin p0, and suppose you specify p0 as the pin name in PSSE. Because driver definition mode decomposes the pin group, psA is listed as one of the PSets available for pin p0. But, because sheet definition mode does not decompose pin groups, specifying p0 does not make psA available. In sheet definition mode, you must specify the pin group pg1 to make psA available.

In general, then, sheet definition mode requires an exact match between the pin name or group that you specify and the pin name or group on which the PSet or signal is defined. Driver definition mode allows more flexible matching.

Note that there can be complications if you specify a pin when the PSet or signal is defined for a pin group: this can happen only in driver definition mode, when pin groups are decomposed. Using the example above, suppose you specify pin p0 and bring up PSet psA, which is defined for the pin group pg1. If you edit the PSet for that pin and then save the change to the PSets sheet, the PSSE adds a row for the pin p0 to the psA PSet definition, which now contains rows for the pin group ppg1 and for the pin p0. This will cause a validation error. For more information, see [Updating Definitions](#).

In addition to pins and pin groups, you can specify a comma-separated list of pins and pin groups. (If you select multiple pins and groups in the Pin/Group Selector, the result is a comma-separated pin list.) You can define a PSet or signal on a list of pins, that is, with a comma-separated list of pins and pin groups in the Pin column. If you specify such a list in PSSE, you are trying to match items that have been defined using the same list, although driver definition mode is again more flexible on this:

- In sheet definition mode, a list can match an item whose Pin column is a comma-separated list of the same pins. It can also match an item whose Pin column is a pin group that contains exactly the pins in the list you supply. If the list contains two pins, the pin group must contain only those two pins.
- In driver definition mode, the pin lists and pin groups on which items are defined are decomposed. For this reason, the pin list you supply can match any subset of the pin lists and pin groups on which items are defined. If an item is defined for a pin group with pins p1, p2, and p3, the pin list "p1,p2" will match the item, because the list is a subset of the pins in the group.

<		>	
	2015 Teradyne, Inc. All rights reserved.		

tdtools.2.24

<

>

TDE Tools : PSet and Signals Editor : Viewing PSets and Signals

Viewing PSets and Signals

This section describes viewing PSets or signals that you have selected for display.
For more information, see:

- [Specifying PSets and Signals for Display](#)
- [Editing and Updating PSets and Signals](#)

Sheet and Driver Definitions

You can define PSets and signals in two ways:

- [On a PSets sheet and Signals sheet in DataTool](#)
- [Dynamically using VBT](#)

Using VBT, when a VBT statement is executed, the definition is loaded into PSet or signal memory: this is known as the "driver definition." In addition, when the workbook is validated, the sheet definition is also loaded into memory, so that the sheet definition becomes part of the driver definition as well.

The PSSE can display either the sheet definition or the current driver definition, depending on the setting of the radio button:

Sheet definition	Includes only definitions found on the PSets or Signals sheet.
------------------	--

Current driver definition Includes both sheet definitions that have been loaded into memory on workbook validation and VBT definitions that have been loaded into memory on execution of the VBT statements.

Controls for Specifying an Item

If no sites are active (for example, if the program is not stopped at a breakpoint), only the sheet definition can be displayed. If sites are active because the program is stopped at a breakpoint, by default the driver definition is displayed.

If an item is defined both on a sheet and in the driver, and if sites are active, you can use the radio button to switch between the definitions. Once you switch between definitions, any differences between the definitions are shown in magenta text:

Name	dcvione
Mode	Voltage
Voltage	0.4
Voltage Range	2 V
Current	0.0000025
Current Range	200 uA
Compliance Range Positive	30 V
Compliance Range Negative	30 V
Condition Bit Mode Change	TRUE
Condition Bit Capture Complete	FALSE
Condition Bit	...

Differences between Sheet and Driver

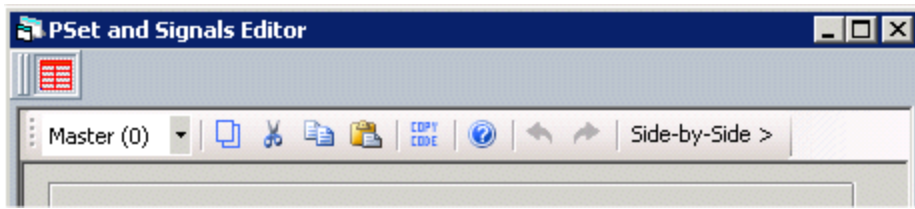
Differences can occur if VBT has changed the value of an item's parameter in the driver after it was loaded from the sheet. They can also occur if the sheet definition has left a parameter blank, which indicates to use the default, or if it selects Auto, which indicates to select the best value. In these cases, the driver definition must show the actual value that is used.

In some cases switching between definitions is not straightforward. In these cases, the PSSE asks you to confirm your action. For example, you may try to switch from the driver to the sheet definition, but the item is not defined on the sheet because it was created dynamically in VBT. Or you may try to switch from the sheet to the driver definition, but modifications made in VBT may

have made the driver definitions inconsistent. In each case, the PSSE warns you and asks for confirmation that you want to make the switch and lose your changes or definitions.

Site-Specific Values

The current driver definition is always site specific, both for viewing and for updating. Use the Select site control on the PSSE toolbar to specify the site to be viewed or updated:



PSSE Site Control

Selecting Master means to use the site selected by the TDE Master Site control.

Viewing Wave Definitions

If you are viewing a signal, one field is Wave Def or Data. If you right-click in this field, the menu contains the option Edit. This brings up the WaveDesigner and WaveScope tools to let you view and edit the wave definition for the signal.

			
	2015 Teradyne, Inc. All rights reserved.		

tdtools.2.25



TDE Tools : PSet and Signals Editor : Editing and Updating PSets and Signals

Editing and Updating PSets and Signals

This section describes editing PSets or signals that you have selected for display.

For more information, see:

- [Specifying PSets and Signals for Display](#)
- [Viewing PSets and Signals](#)

Editing Items

You can edit parameter values for displayed PSets or signals. If a parameter has a finite set of values, the field has a drop-down. Otherwise you edit the field directly.

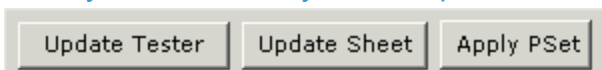
Once you change a parameter value, it appears as blue text until you save the change by updating the sheet or the tester. In addition, if you have selected Current driver definition, the Update Tester button has an asterisk on it to indicate that there are unsaved changes.

You can select one or more fields and cut, copy, and paste values. This can be useful if you have multiple items displayed and you want to copy a set of values from one to the other.

If you change a definition and then, without saving the changes, you switch from Sheet definition to Current driver definition or vice versa, you are asked to confirm that you want to discard the changes. You must save the changes before you can switch definitions.

Updating Definitions

Once you edit items, you can update them using the buttons at the bottom of the PSSE:



PSSE Update Buttons

If Source: is Signal, then the Apply PSet button becomes Load Signal Settings. There are equivalent commands for all these buttons on the PSSE menu.

[Update Sheet](#) uses the currently displayed definitions to update the PSets or Signals sheet. If the appropriate sheet does not exist, it is created. If the displayed PSets or signals do not already exist on the sheet, they are created; otherwise, the existing definitions are updated.

Definitions are not validated before they are used to update the sheet. Once you update the sheet, you must revalidate in DataTool before you can execute or resume executing a program. Once validation occurs, it may indicate places where you must edit the saved definition.

Note a particular case involving pin groups. A PSet or signal may be defined on the sheet for a pin group, but you may choose to display it in the PSSE for a single pin in the group. For example, assume PSet psA defined for pin group pg1, of which pin p0 is a member. You can specify pin p0 and display psA for just that pin. If you edit the definition and then update the sheet, the PSSE cannot determine if you want to update just pin p0 or the entire pin group pg1. Therefore, instead of changing the definition for the entire pin group, it adds a new row for p0 to the PSet definition (which is defined for the pin group pg1), even though this will cause a validation error, because the definition cannot contain rows both for a pin group and for a pin in the group. The validation error calls attention to the fact that you need to edit the definition so that it conforms to your intentions.

[Update Tester](#) updates the driver software definitions of all displayed items. It is available when sites are active. Note that Update Tester does not actually apply the definition to the hardware: for that, you use the third button, Apply PSet/Load Signal Settings. Update Tester also does not permanently save the changes to the workbook sheet; for that, use Update Sheet.

Driver definitions are site-specific. You can update the currently selected site or all sites. Use this button  or the command PSSE>Apply to all sites to toggle between applying to all sites (button selected) or only to the current site (button not selected). On specifying the site, see [Site-Specific Values](#).

When you click Update Tester, the values are validated. If a value is invalid, the cell turns pink, with the text in blue to indicate a changed value. If you place the pointer over the cell, the tooltip displays the error condition and sometimes shows the valid values. If a cell has multiple errors, they are shown in the error window at the bottom of the PSSE. If a validation error occurs, the tester is not updated.

[Apply PSet/Load Signal Settings](#) loads the software definition onto the instrument hardware registers. The button becomes available after Update Tester has been used. That is, you must update the software before you can apply the settings to the hardware.

			
2015 Teradyne, Inc. All rights reserved.			

tdtools.2.26

	
---	---

TDE Tools : PSet and Signals Editor : Creating, Copying, and Saving PSets and Signals

Creating, Copying, and Saving PSets and Signals

You can create PSets or signals by adding a new PSet or signal or by making a copy of a currently displayed PSet or signal.



To add a new item, use this button or the command PSSE>New PSet or Signal. A box appears asking for a name, which must start with a letter and contain only alphanumerics and underscores. A column is added for the new item, with all blank fields.



To copy an existing item, use this button or the command PSSE>Duplicate PSet or Signal. To copy an item, you must have only one item displayed; otherwise the button and command are disabled. A box asks for a name, which must start with a letter and contain only alphanumerics and underscores. A message then asks you to confirm that you want the item added to the appropriate sheet or driver, depending on whether Sheet definition or Current driver definition is selected; the sheet or tester is updated without any additional confirmation.

To save a new or copied item permanently to the workbook, you must use Update Sheet and then save the workbook. Update Tester updates the driver definition, but holds the new item only during the current DataTool session; when you close the workbook, the new item is lost. To have the item available when you close and re-open the workbook, you must use Update Sheet. See [Updating Definitions](#).

	
---	---

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

tdtools.2.27

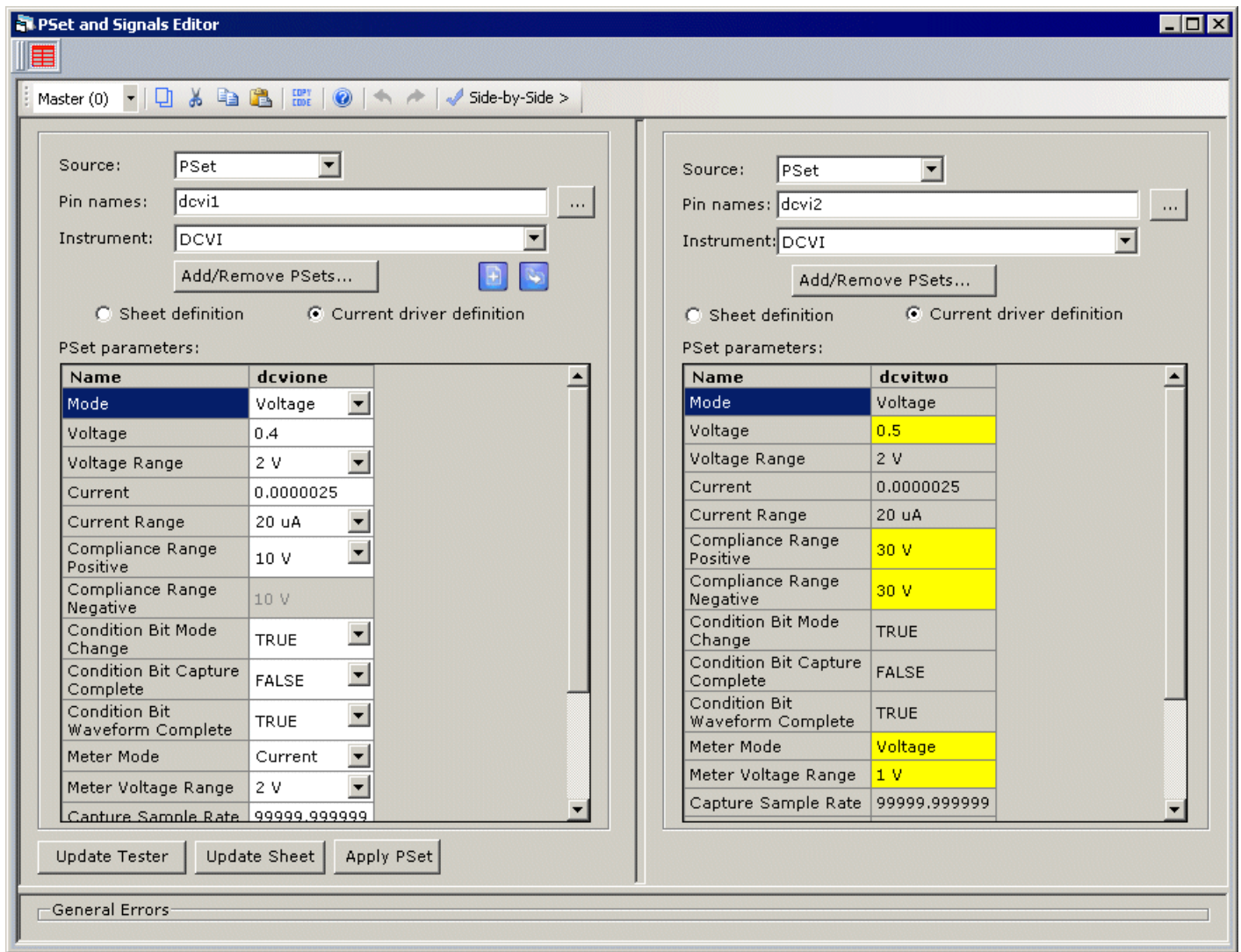


TDE Tools : PSet and Signals Editor : Displaying PSets and Signals Side-By-Side

Displaying PSets and Signals Side-By-Side

The PSSE lets you display and edit up to 10 PSets or signals that have been defined for the same pin and instrument. The PSSE also lets you bring up an additional read-only window where you can display an addition 10 items. This lets you display more items for a pin and instrument, but the side-by-side display is especially useful when you want to compare dissimilar items: for example, PSets and signals defined for the same pin, or PSets or signals defined for different pins or for the same pin but different instruments.

The following figure shows a side-by-side display of PSets defined for two pins of the same instrument type, where the yellow highlights the differences between the two PSets:



Side-by-Side PSSE Display

You can display up to 10 PSets or signals in each window. Item displays on the right are read-only. You can edit items on the left, but not on the right. In addition, commands on the PSSE menu apply only to the display on the left, not on the right.

Side-by-Side >

To use this display, use this button or the PSSE>Side-by-Side command, which toggle between one display and the side-by-side display. In the right side display, specify the source, pin, and instrument as for the regular display, and then select items to display. As on the regular display, you can select up to 10 items.

If items displayed side-by-side are of the same type (PSets or signals) and if their pins are of the same instrument type (so that they have the same parameters), then yellow is used to highlight the parameters of the item on the right that differ from those of the item on the left. If multiple items are displayed on either or both sides, the yellow appears only in the left-most item on the right side, and it indicates differences from the left-most item on the left side.



	2015 Teradyne, Inc. All rights reserved.	
--	--	--

tdtools.2.28

<	>
-----------------	-----------------

TDE Tools : PSet and Signals Editor : Generating VBT Code

Generating VBT Code

Once you create a new PSet or signal or edit an existing one, you can generate VBT code that is based on the current state of the display. This feature is similar to the code-generating features of the debug displays. This section describes the feature as implemented by the PSSE. For more information, see Producing Code from Debug Displays.

When generating VBT code, you may want to use the Code Viewer tool, which displays the most recently generated code, as well as a history of all code generated in the current session. You can copy the code from the Code Viewer into a program, edit the code in the Code Viewer, or save the code to a file. The generated code is always copied to the system clipboard, so you do not need to use the Code Viewer if you want to paste the code directly into a program. But the Code Viewer is useful for viewing, editing, and saving the code. For more information, see Code Viewer.

The following are the general steps for generating code:

1.

You must be stopped at a breakpoint to generate code.
2.

To use the Code Viewer to view, edit, or save code, bring it up now. On the TDE menu bar, use TDE>Tools>Code Viewer.
3.

Specify whether you want the generated code to use the VB With structure. The command PSSE>Code Settings>With Block toggles this option. If you insert a With block into a running program, in some cases VBA requires a reset. To avoid this, generated code in With block format is commented out, so you can insert it without causing a reset. See the note on the "With Block" Option.

4. Specify the kind of code you want to produce. Options, under PSSE>Code Settings, are PSet/Signals Language and New PSet/Signals. These options are described below.
5. Set up the PSSE with the PSets or signals for which you want to generate code. Code is generated for all displayed items (not in the right side of the side-by-side display). Make sure the state of the display represents the state you want created by the code. For example, if you used the copy function to create a new item, you may want to remove the display of the original item so that the code will create only the new item.
6.  Copy the code using this button on the PSSE toolbar (or the keyboard equivalent Ctrl+Shift+C).
7. You can continue to make changes to the displayed items and copy the resulting code. In the Code Viewer, the History View will contain all code generated during the session.

The code is copied to the system clipboard. Even if you do not have the Code Viewer open, you can paste the code from the system clipboard directly into the Visual Basic editor.

If you do have the Code Viewer open, the code is copied there as well as to the system clipboard. For more information, see Code Viewer.

The PSet or signal for which you have created code is not added to the driver until you press Update Tester in the PSSE or until you copy the code into VBT and then execute the copied code.

Options for the kind of code created are New PSet/Signals and PSet/Signals Language.

New PSet/Signals generates VBT statements for the parameters and also generates a PSets.Add or Signals.Add statement before the other statements.

For example, if two Psets are showing and "With Block" is selected:

```
' TheHdw.DCVI.Pins("dcvi1").Psets.Add "pset4"
' TheHdw.DCVI.Pins("dcvi1").Psets.Add "pset5"
' With TheHdw.DCVI.Pins("dcvi1")
'   With .PSets("pset4")
'     .Mode = tIDCVIModeCurrent
'     .Voltage = 8
'   ...
' End With
' With .PSets("pset5")
'   .Mode = tIDCVIModeVoltage
'   .Voltage = 10
' ...
```

```
' End With
' End With
```

PSet/Signals Language does not include an Add statement. Instead, it simply generates the VBT statement for the parameters.

For example, the code shown above looks like this:

```
' With TheHdw.DCVI.Pins("dcvi1")
'   With .PSets("pset4")
'     .Mode = tlDCVIModeCurrent
'     .Voltage = 8
'     ...
'   End With
'   With .PSets("pset5")
'     .Mode = tlDCVIModeVoltage
'     .Voltage = 10
'     ...
'   End With
' End With
```

tdtools.2.29

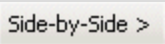
<

>

TDE Tools : PSet and Signals Editor : PSSE Buttons and Commands

PSSE Buttons and Commands

Most buttons on the PSSE toolbar and in the window have equivalent commands on the PSSE menu in TDE. The following commands are available

Code Settings	
> PSet/Signals	
Language	
> New PSet/Signals	
> With Block	
Apply to all sites	
	Toggles between applying updates to all active sites or only to the site specified by the site selector. See Updating Definitions.
Side-by-side	
	Toggles between single window display and side-by-side display, which can display items for different pins or PSets and signals for the same pin. See Displaying PSets and Signals Side-By-Side.
New PSet Or Signal	
	Adds a column with blank fields for a new PSet or signal. See Creating, Copying, and Saving PSets and Signals.
Duplicate PSet Or Signal	
	Creates a copy of an existing PSet or signal in a column with a new name. This command or button is enabled only when there is a

single item displayed. See [Creating, Copying, and Saving PSets and Signals](#).

Sheet definition

 Sheet definition

Displays the currently selected PSets or signals as they are defined on the appropriate sheet in the workbook. See [Sheet and Driver Definitions](#).

Current driver definition

 Current driver definition

Displays the currently selected PSets or signals as they are defined by the instrument driver in IG-XL. See [Sheet and Driver Definitions](#).

Update Tester

Update Tester

Updates the driver software definitions of all the displayed items. It is available when sites are active. Note that this button does not update the hardware. It also does not permanently save changes to the workbook. See [Updating Definitions](#).

Update Sheet

Update Sheet

Uses the currently displayed definitions to update the PSets or Signals sheet. See [Updating Definitions](#).

Apply PSet
Load Signal Settings

Apply PSet

Load Signal Settings

Loads the software definition onto the instrument hardware registers. If Source: is PSets, Apply PSet is available; if it is Signal, Load Signal Settings is available. See [Updating Definitions](#).

tdtools.2.30

<	>
-----------------	-----------------

TDE Tools : Results Tool

Results Tool

 This is the icon for the Results tool.

The Results tool allows you to view a subset of datalogged results without having to go to the output window or to a file. Use Results with MiniFlow.

The following topics are included:

- | | |
|---|---------------------|
| • | Bringing Up Results |
| • | Results Displayed |
| • | Updating Results |

Bringing Up Results

You can bring up Results when you are stopped at a test on MiniFlow. Use TDE>Tools>Results. It displays the results of the most recently executed test.

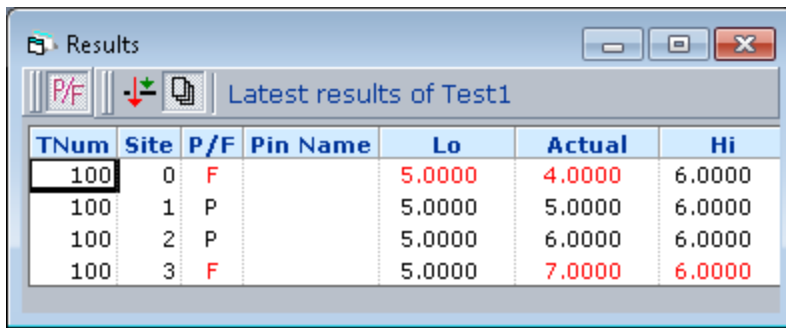
Once execution has run to the end of the flow, Results no longer displays any results.

Results Displayed

For the selected test instance, Results displays the results of the limits comparisons made for the test. If the test performs more than limits test, each result has its own row with a distinct TNum value.

In each row, Results displays the test number, site, and pass/fail results. The remaining columns display additional information:

- | | |
|---|--|
| • | For a parametric test, they show low and high limits and the result value: |
|---|--|



TNum	Site	P/F	Pin Name	Lo	Actual	Hi
100	0	F		5.0000	4.0000	6.0000
100	1	P		5.0000	5.0000	6.0000
100	2	P		5.0000	6.0000	6.0000
100	3	F		5.0000	7.0000	6.0000

Results Window

For a failing test, the actual result and the limit that caused the failure are both shown in red.

- For a functional test, they show the pattern executed, the first fail cycle, the failing vector, and the number of failing vectors.

If the datalog for a result would have "N/A" for the first fail cycle or the total number of fails, Results displays 0 for the value. In this case, "0" can mean "N/A."

 The Show All Sites command on the Results menu and this button toggle between showing only the Master Site (as defined on the TDE toolbar) and showing all sites.

 The Show Failures Only command on the Results menu and this button toggle between showing only failures and showing all results.

Updating Results

As you step through the tests in MiniFlow, Results updates its display to show the most recently executed test.

If you have stepped into a test procedure or code, Results is updated when execution returns to MiniFlow. It is not updated as you step through procedures or VBT. The update occurs after the entire test has been executed.

	
2015 Teradyne, Inc. All rights reserved.	

tdtools.2.31



TDE Tools : Tester Summary Display

Tester Summary Display

 This is the icon for the Tester Summary Display.

The Tester Summary Display (TSD) is a high-level view of the instruments in the current tester setup. It shows the programmable pins, the instruments connected to the pins, the state of the connection, and the current settings of the most important instrument parameters.

The Tester Summary Display looks like this:

(M) Tester Summary Display

Pin	Instrument	Pin Type	DIB Chan	Chan Type	Connect	Mode	Voltage	Current	Gate	Period	Voh	Vol	Vih	Vil	Amplitude	Sample Rat
Src1	PPMU	Analog	22.b413	BBACSrcPos	N	ForceVMeasureI	-45.777 μ V	30.518 nA	Y							
Src1	BBACSource	Analog	22.b413	BBACSrcPos	N										1.000 V	10.000 MHz
Cap1	PPMU	Analog	22.b405	BBACCapPos	N	ForceVMeasureI	-45.777 μ V	30.518 nA	Y							
Cap1	BBACCapture	Analog	22.b405	BBACCapPos	N											1.000 MHz
vdd	DCVI	Power	16.s402	DCVI	Y	Voltage	4.000 V	200.000 mA	Y							
osc1	PPMU	Input	1.d413	I/O	N	ForceVMeasureI	0V	0A	Y							
osc1	Digital	Input	1.d413	I/O	Y					sec	4.800 V	600.000 mV	4.000 V	800.000 mV		
osc2	PPMU	Output	1.e406	I/O	N	ForceVMeasureI	0V	0A	Y							
osc2	Digital	Output	1.e406	I/O	Y					sec	4.800 V	600.000 mV	0V	0V		
nsireq	PPMU	Output	1.p304	I/O	N	ForceVMeasureI	0V	0A	Y							
nsireq	Digital	Output	1.p304	I/O	Y					sec	4.800 V	600.000 mV	0V	0V		
nemul	PPMU	Input	1.k309	I/O	N	ForceVMeasureI	0V	0A	Y							
nemul	Digital	Input	1.k309	I/O	Y					sec	0V	0V	4.000 V	800.000 mV		
ps_meas	PPMU	I/O	1.d404	I/O	N	ForceVMeasureI	0V	0A	Y							
ps_meas	Digital	I/O	1.d404	I/O	Y					sec	4.800 V	600.000 mV	4.000 V	800.000 mV		
vdd2	DCVI	Power	16.w402	DCVI	Y	Voltage	4.000 V	20.000 mA	Y							

Tester Summary Display

Each pin name and instrument name in the display is a link that can bring up the debug display for that instrument and pin. You can therefore use this top-down view to get a summary of the entire tester configuration and then navigate to specific instrument debug displays for more detailed instrument setup information.

The Tester Summary Display is read-only. To change an instrument parameter, you must bring up the appropriate debug display.

The following information is available:

- Using the Display

•	Configuring and Using Views
•	Default View
•	View Configuration Window
•	Buttons and Commands

tdtools.2.32



TDE Tools : Tester Summary Display : Using the Display

Using the Display

This section describes basic use of the Tester Summary Display. See also [Configuring and Using Views](#).

Bringing Up the Display

To use the Tester Summary Display:

- | | |
|----|---|
| 1. | Run to a breakpoint on the Flow Table or in VBE. |
| 2. | On the IG-XL Tools tab , in the Launch group, click Tools and select Tester Summary Display. You can also launch the tool from TDE. |

You can launch multiple instances of the Tester Summary Display. Each instance is independent of the others. This lets you configure each display differently. See [Configuring and Using Views](#).

Selecting a Site

The Tester Summary Display shows the summary for one site at a time. The site selector on the display's toolbar lets you select a site. You can also select Master, which is the site currently selected by the TDE toolbar site selector.

The current site is displayed in the window title, as "(n) TesterSummaryDisplay," where n is the site number. "(M)" indicates that the master site is selected.

If the selected site is inactive, no data is displayed. The message bar indicates that the current site is inactive.

Refreshing the Display

The Tester Summary Display is automatically refreshed whenever the hardware state changes while the program is running in debug mode. You can also use the command `TDE>Refresh` or the F12 key.

< >		
	2015 Teradyne, Inc. All rights reserved.	

tdtools.2.33



TDE Tools : Tester Summary Display : Configuring and Using Views

Configuring and Using Views

In addition to its default view of information, the Tester Summary Display (TSD) lets you configure views that are tailored to your specific needs. A view determines what information is displayed and how it is displayed. Once you configure a view, you can save it and apply it later.

Configuring a View

The View Configuration window initially has the settings of the current view: either the default view or a previously saved view that you have applied.

To configure a view:

1.  Use this button on the TSD toolbar or the Configure View command on the View or right-click menu.

This brings up the View Configuration window, which lets you specify nondefault settings.

2. When you specify the settings you want, click Apply to apply the changes without quitting the window or OK to apply them and close the window.

Many controls on the View Configuration window have equivalents in buttons, commands, or mouse movements. These equivalents are noted in the description of the View Configuration window. Settings selected with these equivalents can be saved to a view.

Saving a View

 To save the view configuration, use this button on the TSD toolbar or the Save Current View Configuration command on the View or right-click menu. This brings up a window that lets you name the view. It saves any changes made using the View Configuration window or any of the button, command, or mouse equivalents.

 Once you save a view, you can apply this view using this button on the TSD toolbar or the **Apply** a View command on the View or right-click menu. Select a view from the drop-down list that appears.

Deleting a View

 To delete views, use this button on the TSD toolbar or the **Delete Views** command on the View menu. Check the views you want to delete.

Returning to the Default View

 At any point while you are making changes, you can return to the default view by using this button on the TSD toolbar or the **Default View** command on the View or right-click menu. For more information on views see:

- [Default View](#)

- [View Configuration Window](#)

<

>

2015 Teradyne, Inc. All rights reserved.

tdtools.2.34

<	>
-----------------	-----------------

TDE Tools : Tester Summary Display : Default View

Default View

On startup, the Tester Summary Display loads the state of the tester using a default view. This section describes the settings of the default view and indicates how to change the default settings.

 At any point while you are changing the current view, you can return to the default view by using this button on the TSD toolbar or the Default View command on the View or right-click menu.

The default view:

- | | |
|--|---|
| | <ul style="list-style-type: none">Shows all programmable pins in the channel map. You can hide specific pins. The display shows only individual pins, not pin groups, although you can use pin group names to specify which pins to hide or show. On hiding pins, see Pins Tab. |
| | <ul style="list-style-type: none">Shows only pins mapped to instruments that have displayable parameters. See Options Tab. |
| | <ul style="list-style-type: none">Shows all instruments, connected or disconnected. You can specify only connected or only disconnected instruments. See Options Tab. |
| | <ul style="list-style-type: none">Sorts entries by pin names as they appear in the pin map. You can change to ascending or descending order by a specific column. See Options Tab. |

The Tester Summary Display has two kinds of columns:

- | | |
|--|--|
| | <ul style="list-style-type: none">Columns applicable to all instruments (see Common Columns) |
|--|--|

- Instrument-specific columns (see Instrument-Specific Parameters)

A column appears only if at least one instrument has a value for it. For example, the :DA column is blank if the pin has not been programmed as a DIB access pin; it is Y if it is so programmed. If no pin has a Y, the column would be blank, so the column is not displayed. Similarly, if you hide an instrument that is the only one with a value for an instrument-specific parameter, the column for that parameter is not displayed.

The default view includes all common and instrument-specific columns, except those that would be blank. You can hide columns using the Options tab or the Hide Column command on the right-click menu.

Common Columns

Common columns apply to all instruments. In the default view, these are the leftmost columns. This is information from the pin map and channel map, along with an indication whether the instrument is connected.

Pin

The pin name as programmed on the pin map.
A pin name appears twice if a secondary instrument or multiple resources are behind the pin, or if the pin is programmed for DIB access or combined connection. See the descriptions of the Instrument, :DA, :M, and :C columns.
The pin name is a blue link that brings up the debug display for the instrument set to this pin. If a pin is listed twice, the link brings up the display for the instrument in the row where you click.

Instrument

All instruments mapped to the pin on the channel map. If a secondary instrument is behind the pin, the display includes an entry for each instrument.
The instrument name is a blue link that brings up the debug display for this instrument set to this pin. The entries in each row for Pin and Instrument bring up the same debug display.

Pin	Instrument	Pin Type	DIB Chan	Chan Type
awq1	PPMU	Analog	8.c104	AWGPos
awq1	AWG	Analog	8.c104	AWGPos

Pins with Secondary Instruments

Pin Type

The pin type as programmed on the pin map.

DIB Channel	The channel as programmed on channel map. By default, the DIB channel is displayed in Pogo pin format. You can use the Options tab of the View Configuration window to specify signal format instead. See Options Tab.
Chan Type	The channel type as programmed on the channel map.
:DA	Whether the pin was programmed as a DIB access pin on the channel map. Y indicates a DIB access pin. Otherwise the column is blank.

Pin	Instrument	Pin Type	DIB Chan	Chan Type	:DA
dc75	DCVI	Analog	17.d402	DCVI	
dc75	TurboACCCapture	Analog	5.a304	TurboACCCapPos	Y

Pin with DIB Access

:M	Whether the pin was programmed as a multiple resource pin in the channel map. Y indicates a multiple resource pin. Otherwise the column is blank.
----	---

Pin	Instrument	Pin Type	DIB Chan	Chan Type	:M
dcviMultiple	DCVI	Analog	16.s408	DCVI	
dcviMultiple	DCDiffMeter	Analog	16.s408	DCDiffMeter	Y

Pin with Multiple Resources

:C	Whether the pin was programmed as a combined connection pin in the channel map. Y indicates a combined connection pin. Otherwise the column is blank.
----	---

Pin	Instrument	Pin Type	DIB Chan	Chan Type	:C
dcvsVsm4	DCVS	Analog	29.x202	DCVSMerged2	
dcvsVsm4	DCVS	Analog	30.x202	DCVSMerged2	Y

Combined Connection Pin

Connect	Whether the instrument is connected to the DUT: Y or N.
---------	---

Instrument-Specific Parameters

Some columns display instrument-specific parameters. In the default view, these are the columns to the right of Connect.

Only major parameters are displayed for each instrument. The columns in the default display consist of all these parameters for all instruments. If two instruments have the same parameter

name, the same column is used for both instruments. If a column does not apply to an instrument, the cell is blank, and a tooltip indicates that this parameter is not displayed for this instrument. If you hide an instrument or pin that has the only value for an instrument-specific column, the column is hidden because otherwise it would be blank.

<		>	
	2015 Teradyne, Inc. All rights reserved.		

tdtools.2.35

<	>
-----------------	-----------------

TDE Tools : [Tester Summary Display](#) : View Configuration Window

View Configuration Window

The following sections describe the features of the [View Configuration](#) window. These topics are included:

- [Pins Tab](#)
- [Pin Types Tab](#)
- [Instruments Tab](#)
- [Options Tab](#)

For more information, see:

- [Configuring and Using Views](#)
- [Default View](#)

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

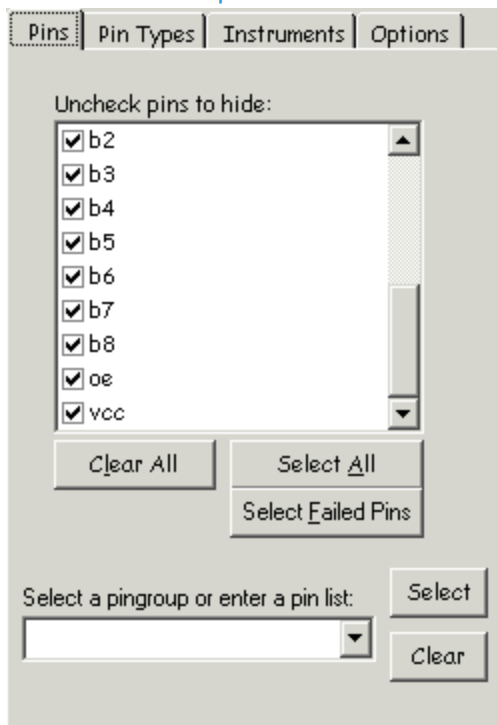
tdtools.2.36



TDE Tools : Tester Summary Display : Pins Tab

Pins Tab

The Pins tab is part of the View Configuration window. The Pins tab selects the pins to be displayed:



Pins Tab

Only pins that are checked are displayed. The others are hidden. Once you select the pins you want, click OK or Apply to apply the currently selected pins.

The list contains all pins in the pin map that are connected to instruments.

The selections you make are additive, that is, they are added to the pins already selected. You can start by clicking Clear All to clear all pins before making selections.

You can select individual pins in the window. You can use Clear All or Select All and then check or uncheck individual boxes to specify the pins you want.

To display only failed pins, click [Clear All](#) and then [Selected Failed Pins](#). This option is also available as a button on the Tester Summary Display toolbar.

Controls at the bottom of the tab let you select multiple pins at once. You specify the pins you want and then click [Select](#). These pins are then added to the pins already selected in the top window. The options are:

- You can enter a comma-separated list of pin names and then click [Select](#).
- The drop-down list contains the pin groups defined in the pin map. You can select a pin group and then click [Select](#). This checks the boxes for the pins in the group.
- You can enter a template with wildcards to match certain pin names. An asterisk (*) matches any number of characters, including 0. A question mark (?) matches exactly one character. Enter the template and then click [Select](#).
- Because selections are additive, you can combine or repeat these options to get the set of pins you want.
- You can enter any of these options and then click [Clear](#) to hide the pins matched by what you have entered.

Some controls on this tab have equivalents on the main display:

- To hide a pin, right-click a cell in the pin row and select [Hide Pin](#).
-  To display only failed pins, use this button on the TSD toolbar or [View>Show Failed Pins Only](#).

For more information, see:

- [Configuring and Using Views](#)
- [Default View](#)

<		>	
	2015 Teradyne, Inc. All rights reserved.		

tdtools.2.37

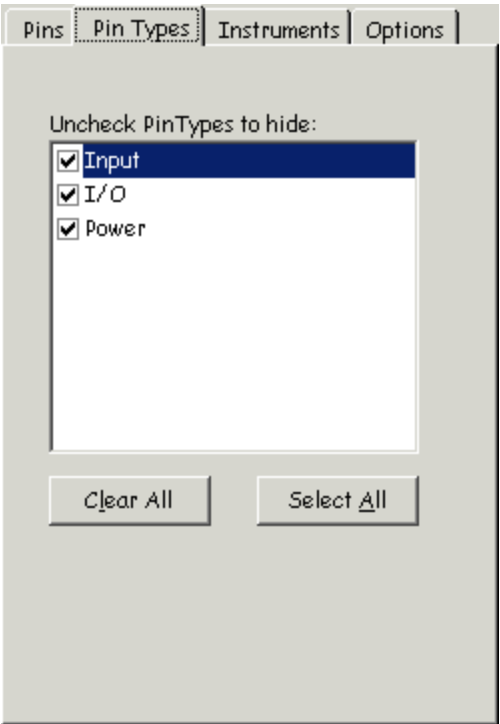
<

>

TDE Tools : Tester Summary Display : Pin Types Tab

Pin Types Tab

The Pins Types tab is part of the View Configuration window. The Pin Types tab selects pin types, as defined on the pin map, to be displayed:



Pin Types Tab

The list contains all pin types that are used on the pin map by the pins included in the display. On the main display, you can hide all pins of a type by right-clicking a cell in a row that has the pin type and selecting Hide Pin Type. For more information, see:

- [Configuring and Using Views](#)

•	Default View
---	------------------------------

	
---	---

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

tdtools.2.38

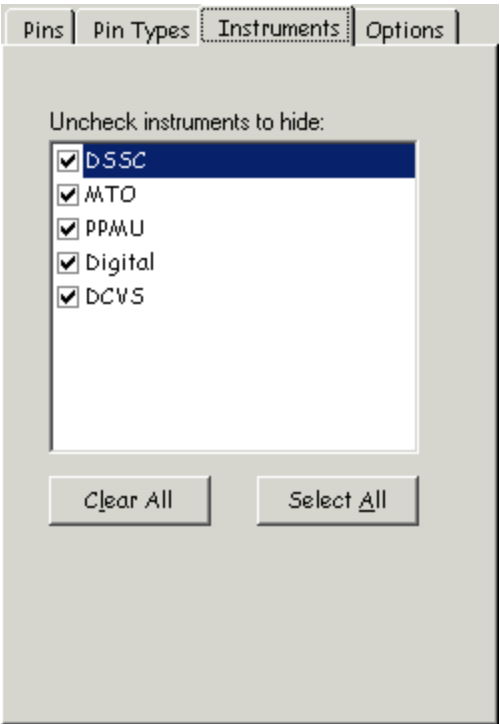
<

>

TDE Tools : Tester Summary Display : Instruments Tab

Instruments Tab

The Instruments tab is part of the View Configuration window. The Instruments tab selects the instruments to be displayed:



Instruments Tab

The list contains the instruments that are connected to pins on the channel map, as well as any secondary instruments, such as PPMU and RefGround.

On the main display, you can hide all instruments of a type by right-clicking a cell in a row that has the instrument and selecting Hide Instrument.

For more information, see:

- [Configuring and Using Views](#)

•	Default View
---	------------------------------

	
---	---

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

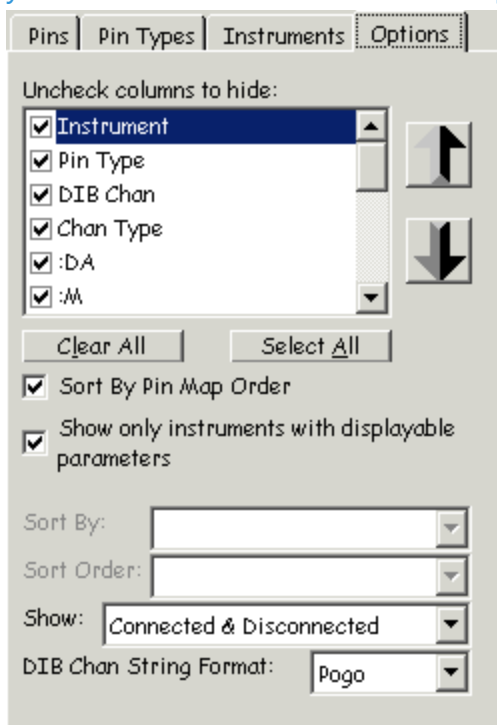
tdtools.2.39



TDE Tools : Tester Summary Display : Options Tab

Options Tab

The Options tab is part of the View Configuration window. The Options tab lets you specify how you want the information to be displayed:



Options Tab

The window at the top lets you specify which columns to display or hide. The list displays all of the standard columns that are applicable to all instruments, plus all instrument-specific columns. The list includes even those columns that will not be displayed because they have no values; checking or unchecking the box for such columns has no effect.

You cannot hide the Pin column, so it does not appear on the list.

Use the up and down arrows to the right of the window to rearrange the order of the columns in the display. Select a column by clicking it; this is different from checking the check box. Then use

the up arrow to move the selected column to the right in the display and the down arrow to move it to the left. Note that you cannot move the Pin column from the leftmost position.

By default, the Sort by pin map order check box is checked, and pins are displayed in the order in which they appear in the pin map. If you uncheck the box, the Sort by and Sort order drop-down controls are enabled. Sort by contains all columns in the display. Select the column to use for sorting. Then, for Sort order, select Ascending or Descending. On the display, a white triangle appears in the column used for sorting, pointed up to indicate ascending order and down for descending order.

By default, the Show only instruments with displayable parameters check box is checked. Some instruments do not have any displayable parameters other than what is in the channel map. Because these instruments would not display useful parameters, the default is not to display them.

By default, the Show drop-down control selects Connected & Disconnected. You can use the control to display only Connected or only Disconnected instruments.

By default, the DIB chan string format selects Pogo. You can use the drop-down to select Signal. The main display has these equivalents to the controls on the Options tab:

- To hide individual columns, right-click in the column and select Hide Column.
- To change the order of columns, left-click a column header and drag it to a new position.
- To sort using a column, left-click the column header. Click the column header again to toggle between ascending and descending order.
- To return to the default sorting, use the Sort By Pin Map Order command on the View or right-click menu.

For more information, see:

- Configuring and Using Views
- Default View



	2015 Teradyne, Inc. All rights reserved.	
--	--	--

tdtools.2.40



TDE Tools : Tester Summary Display : Buttons and Commands

Buttons and Commands

Commands specific to the Tester Summary Display are on the TDE View menu. The Tester Summary Display toolbar has a corresponding button for most commands. In addition, you can use some standard TDE commands, such as Save, with the Tester Summary Display as well. A right-click menu is also available.

View Menu

The following commands are on the TDE View menu. Most have buttons on the Tester Summary Display toolbar, and most are also available on the right-click menu.

Default View	 Restores the default view. Use this command after you apply a nondefault view or otherwise modify a display. See Default View.
Sort By Pin Map Order	Sorts the display using the order of the pin names in the pin map. Default.
Save Current View Configuration	 Saves the current view configuration so you can apply it later. The command brings up a box that lets you name the view.
Configure View	 Brings up the view configuration dialog box to let you specify nondefault view settings. After you configure a view, you can save it.
Apply A View	 Applies a previously saved view configuration. The command brings up a box with a drop-down list of the available views.

Delete Views		Lets you delete previously saved views. The command brings up a box with a list of available views. Check each view that you want deleted.
Show Failed Pins Only		Configures the display to show only pins that have failed a test.

Other TDE Menus

The standard TDE commands on other menus can be used with the Tester Summary Display. These commands also have buttons. Of special use are the following:

File> Save		Saves the contents of the display to a tab-delimited ASCII file. The file does not include hidden columns or rows.
File> Print		Prints the tab-delimited ASCII version of the contents of the display to the specified printer.
Edit> Copy All		Copies the tab-delimited ASCII version of the contents of the display to the system clipboard.
Edit> Find		Finds the specified text in the display. You can select a case-sensitive match or an exact match. Find highlights the matching cells. The Clear Results button clears the highlighting.

Right-Click Menu

Most commands on the View menu are also available when you right-click any cell in the display. In addition, the menu contains options to hide parts of the display based on the value of the selected cell. You can hide the following:

- | | |
|---|---|
| • | The selected pin |
| • | The selected column (except for the Pin column) |
| • | All pins of the type of the selected pin |

- [All instruments of the type connected to the selected pin](#)

	
---	---

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

tdtools.2.41



TDE Tools : VBT Debugger

VBT Debugger



This is the icon for the VBT Debugger.

The VBT Debugger lets you step through VBT code within the context of the TDE window. It provides an integrated debugging environment for VBT code, letting you debug code without switching between windows. You can also use the VBT Debugger with debug displays, so you can step through each line of the VBT code and immediately see its effect in the debug display.

The VBT Debugger offers a subset of the functionality of the full Visual Basic Editor (VBE). It provides the important VBE viewing features, such as the Immediate, Locals, and Watch windows, plus some editing capability.

Enabling the VBT Debugger

You do not explicitly invoke the VBT Debugger. Instead, you enable it so that it can come up at the appropriate time.

To toggle between enabling and disabling the VBT Debugger, use this command:

TDE>Options>Use VBTDebugger.

Entering and Exiting the VBT Debugger

If you have enabled the VBT Debugger, you can enter the Debugger during a Debug Run:

- If you have set a breakpoint in a VBT code module, running to the breakpoint brings up the VBT Debugger in the TDE window. This happens when you do a Debug Run from the Flow Table or from MiniFlow.
- From MiniFlow, if you step into a VBT test, the VBT Debugger comes up in the TDE window.

The VBT Debugger comes up in the same toolgroup window as MiniFlow, because the two tools should not be needed at the same time.

Note that you cannot invoke all TDE tools when a program is stopped at a breakpoint in the VBT Debugger. Specifically, tools that are based on DataTool sheets, such as MiniFlow or Mixed Signal Workshop, are disabled on the TDE menu. If the tool was created before arrival at the breakpoint, it is shown in a disabled state while stopped at the breakpoint.

The VBT Debugger closes when you exit from the VBT test. You can close the VBT Debugger window while stepping through a test. This is not recommended, however, because you must exit from VB before you can continue to use IG-XL tools.

VBT Debugger Capabilities

The VBT Debugger provides a subset of VBE capabilities. The Debugger toolbar has standard VBE buttons for these functions: Run, Break, Reset, Toggle Breakpoint, Step Into, Step Over, Step Out, Locals Window, Immediate Window, Watch, Quick Watch, and Call Stack. VBE keyboard shortcuts for these functions also work within the VBT Debugger.

The VBT Debugger also permits editing in the code pane. There are no buttons to facilitate editing, such as VBE’s Comment Block button. But any editing shortcuts that work in VBE will also work in the VBT Debugger.

<

>

2015 Teradyne, Inc. All rights reserved.

tdtools.2.42

<

>

TDE Tools : Watch

Watch

 This is the icon for the Watch tool.

The Watch tool lets you view the current values for result, bin, and sort as you step through the MiniFlow or MiniBin. You can also specify flags whose states you want to watch.

To bring up the tool, use the TDE command `TDE>Tools>Watch`.

The built-in toolset FlowDebug includes the Watch tool, as well as MiniFlow and FlowCall Stack.

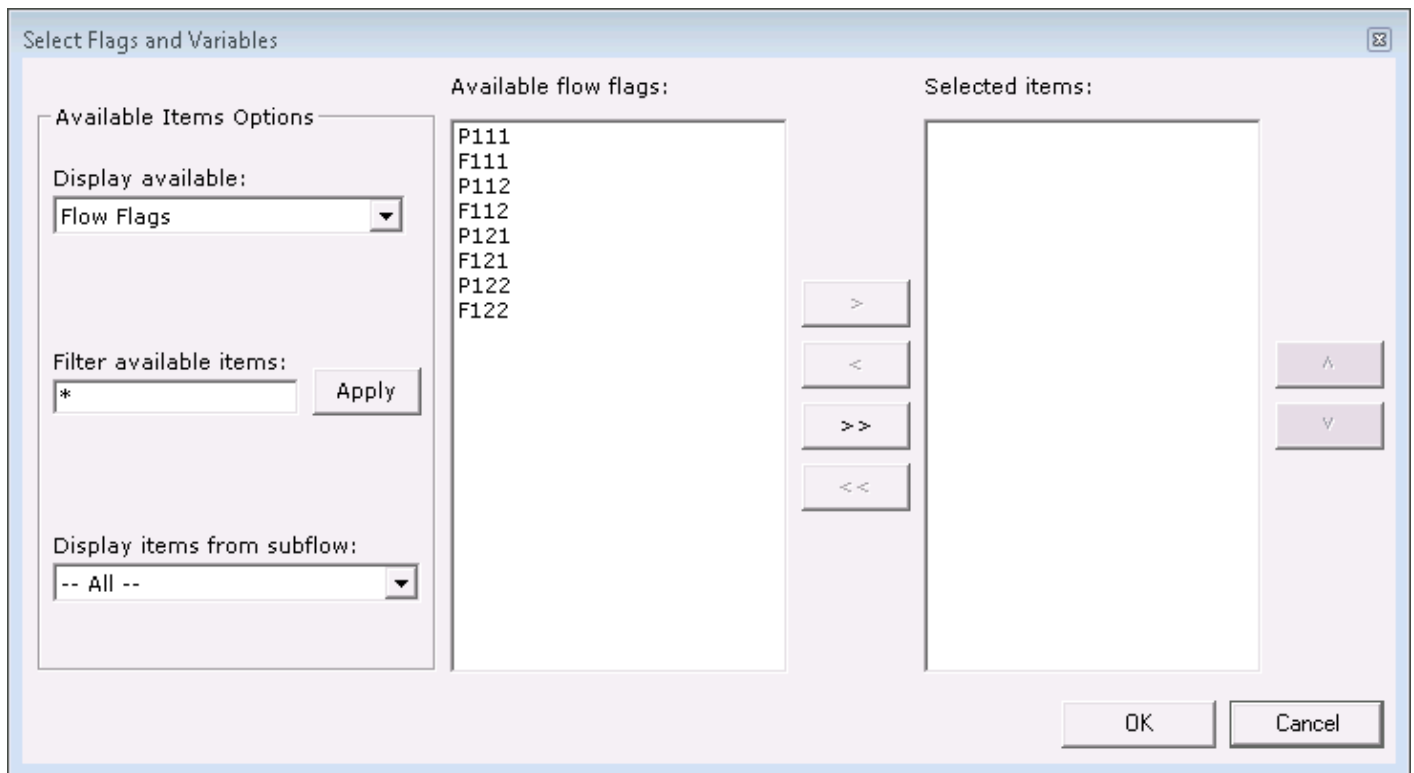
 Use the `Watch>Show All Sites` command or the button to toggle between showing one site (the current Master Site in the TDE toolbar) and showing all sites.

By default, Watch displays the bin, sort, and result:

Name	Site 0	Site 1	Site 2	Site 3
Bin	131	0	122	112
Sort	-1	-1	-1	-1
Result	FAIL	PASS	FAIL	FAIL

Default Watch Window

 You can add specific flags and site variables to the display. Click this button or use `Watch>Select Flags and Variables`. This window comes up:



Select Flags and Variables Window

The Display available: drop-down selects between flow flags and site variables. The Filter available items: box lets you specify a wildcard template. For example, to display only the Pass flags above, use P*. The Display items from subflow: drop-down lets you display the items from all flows and subflows or only from the specified flow.

Select the items you want in the Available panel and use > to move them to the Selected panel or < to move selected items back to the Available panel. Use >> and << to move all items between the panels. In the Selected items: panel, use ^ and v to change the order of the items.

As you step through the flow or bin table, any value that changed during the step is displayed in blue to indicate a change. On the next step, the value is displayed in black.

You can change the state of any flag or site variable in Watch. Double-click the cell and change the value. This lets you observe the effects of different flag states on binning and other result processing.

It is possible to have a large number of flags and variables in Watch, which can make it difficult to see what changed on the most recent flow step:

Name	Site 0	Site 1	Site 2	Site 3
Bin	-1	-1	122	122
Sort	-1	-1	-1	-1
Result	N/A	N/A	FAIL	FAIL
P111	T	T	T	T
F111	F	F	F	F
P112	T	T	T	F
F112	F	F	F	T
P121	T	T	T	F
F121	F	F	F	T
P122	T	T	F	F
F122	F	F	T	T

All Flags in Watch

 The AutoRefresh feature lets you display only the flags and variables that were modified are read in the flow step just executed. Click this button or use Watch>AutoRefresh to enable and disable AutoRefresh.

The following shows that four flags were modified on the just executed flow step:

	Row	OpCode	Parameter	Device
1	5	set-error-bin		
2	6	Test	Test11	
3	7	Use-Limit	Test11	
4	8	Use-Limit	Test11	
5	9	Test	Test12	
6	10	Use-Limit	Test12	
7	11	Use-Limit	Test12	
8	12	Test	Test13	flag=true P121
9	13	Use-Limit	Test13	
10	14	Use-Limit	Test13	
11	15	Test	Test14	
12	16	Use-Limit	Test14	
13	17	Use-Limit	Test14	
14	18	set-device		

Watch

Name	Site 0	Site 1	Site 2	Site 3
Bin	-1	-1	-1	112
Sort	-1	-1	-1	-1
Result	N/A	N/A	N/A	FAIL
F111	F	F	F	F
F112	F	F	F	T
P111	T	T	T	T
P112	T	T	T	F

Displaying Modified Flags

The following shows the one flag that was read as part of evaluating the device qualifier:

1

2

3

4

5

6

7

8

9

10

11

12

13

14

Row	OpCode	Parameter	Device
1	5	set-error-bin	
2	6	Test	Test11
3	7	Use-Limit	Test11
4	8	Use-Limit	Test11
5	9	Test	Test12
6	10	Use-Limit	Test12
7	11	Use-Limit	Test12
8	12	Test	Test13
9	13	Use-Limit	Test13
10	14	Use-Limit	Test13
11	15	Test	Test14
12	16	Use-Limit	Test14
13	17	Use-Limit	Test14
14	18	set-device	

Watch

AUTO

Name	Site 0	Site 1	Site 2	Site 3
Bin	131	-1	122	122
Sort	-1	-1	-1	-1
Result	FAIL	N/A	FAIL	FAIL
P121	T	T	T	F

Displaying Read Flag

<

>

2015 Teradyne, Inc. All rights reserved.